Syrian Arab Republic

Ministry of Higher Education

Syrian Virtual University

# Bachelor of IT Program BAIT

# MS SQL Server Administration IIS404

Chapter four

## Security Fundamentals in MS SQL Server

**Eng. Ayham Mohammad**

Syrian Virtual University SVU – 2020

## Contents

1. Intro ..... 1
2. Authentication vs. Authorization ..... 1
2.1. Authentication in SQL Server ..... 1
2.1.1. Windows authentication ..... 2
2.1.2. SQL Server Authentication ..... 2
2.1.3. Azure Active Directory Universal Authentication ..... 3
2.1.4. Azure Active Directory Password Authentication ..... 3
2.1.5. Active Directory Integrated Authentication ..... 4
2.1.6. Other types of logins ..... 4
2.2. Authentication to SQL Server on Linux ..... 4
2.3. Factors in securing logins ..... 5
2.3.1. Using mixed mode ..... 5
2.3.2. Login security ..... 6
3. Managing Principals ..... 8
4. Role-Based Permissions ..... 8
4.1. Server and DB Roles in SQL Server ..... 8
4.1.1. Fixed Server Roles ..... 9
4.1.2. Fixed Database Roles in SQL Server ..... 10
4.1.3. Database Roles and Users ..... 12
4.2. Solving orphaned SIDs ..... 13
4.3. Ownership versus authorization ..... 16
5. Ownership & User-Schema Separation ..... 18
5.1. User-Schema Separation ..... 18
5.2. Schema Owners and Permissions ..... 18

5.3. Built-In Schemas....19
5.4. The Principle of Least Privilege....20
6. Permission Statements....20
7. IMPERSONATE....21
8. CONTROL SERVER|DATABASE....22
9. Permissions through Procedural Code....22
9.1. Ownership Chains....22
9.2. Procedural Code & Ownership Chaining....22
10. Row-Level Security....23
11. Transparent Data Encryption (TDE)....25
11.1. Using Transparent Data Encryption....25

# 1. Intro

- **Principle: ممكن يكون شخص أو Role , نعطيه سماحية على ال Securable (ممكن ان يكون جدول, او view...)**

A defense-in-depth strategy, with overlapping layers of security, is the best way to counter security threats. SQL Server provides a security architecture that is designed to allow database administrators and developers to create secure database applications and counter threats.

A SQL Server instance contains a hierarchical collection of entities, starting with the server . Each server contains multiple databases, and each database contains a collection of securable objects. Every SQL Server securable has associated permissions that can be granted to a principal, which is an individual, group or process granted access to SQL Server .

# 2. Authentication vs. Authorization

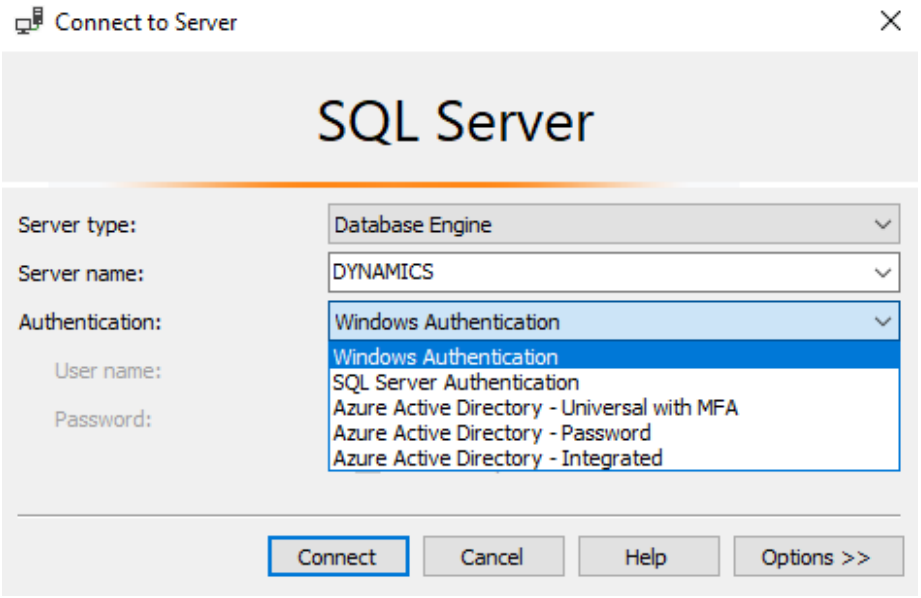
SQL Server security framework manages access to securable entities through authentication and authorization.

- **Authentication : هي عملية التحقق من وجود المستخدم ( ال Principle )**
- **Authorization: هي عملية التخقق من صلاحيات المستخدم على موارد معيّنة**

***Authentication*** is the process of logging on to SQL Server by which a principal requests access by submitting credentials that the server evaluates. Authentication establishes the identity of the user or process being authenticated.

**Authorization** is the process of determining which securable resources a principal can access, and which operations are allowed for those resources.

## *2.1. Authentication in SQL Server*


Connect to Server
SQL Server
Server type:
Database Engine
Server name:
DYNAMICS
Authentication:
Windows Authentication
User name:
Password:
Windows Authentication
SQL Server Authentication
Azure Active Directory - Universal with MFA
Azure Active Directory - Password
Azure Active Directory - Integrated
Connect
Cancel
Help
Options >>

### 2.1.1. Windows authentication

هي الطريقة ال **Default**

عند تحديد إستخدام ال **Windows Authentication** عند تسجيل الدخول لل **SQL Server** , لن يتم طلب **username, password**
بل سيتم إستخدام معلومات التعريف الخاصة بال **user** المستخدم في **windows** ( ينصح بإستخدام هذه الطريقة من قبل **Microsoft** )
و يطلق عليه **integrated security** لأنه **integrated** ( مدمج – يستطيع سحب معلومات ال **users** منه ) مع **windows**

It is the default, and is often referred to as integrated security because this SQL Server security model is tightly integrated with Windows. Specific Windows user and group accounts are trusted to log in to SQL Server.

Windows users who have already been authenticated do not have to present additional credentials. Users are already logged onto Windows and do not have to log on separately to SQL Server. The following SqlConnection.ConnectionString specifies Windows authentication without requiring a user name or password.

هذه ال SQLConnection.ConnectionString تقوم بتفعيل ال Windows Authentication في حال غير مفعّل:

*Server=MSSQL1; Database=IIS404; Integrated Security=true;*

Recommended by Microsoft wherever possible, because:

- It uses a series of encrypted messages to authenticate users in SQL Server.
- When SQL Server logins are used, SQL Server login names and passwords are passed across the network, which makes them less secure

### 2.1.2. SQL Server Authentication

**عند إستخدام هذه الطريقة, تتيح إدخال username, password ( ليس له أي علاقة بالمستخدمين في نظام التشغيل )**
**و تخزّن هذه المعلومات في ال master.db بشكل مشفّر ( لذلك مهم جدا الإهتمام بحماية ال master.db )**

SQL Server Authentication is a method that stores user names and passwords in the master database of the SQL Server instance. SQL DBAs must manage password complexity policy, password resets, locked-out passwords, password expiration, and changing passwords on each instance that uses SQL Server Authentication.

عند إنشاء user يتم تعريف Security Id له SID, ( هي التي يتعرف عليها ال SQL Engine و يستخدمها و ليس الإسم العادي )
و تكون غير ال Principle ID التي تنشأ أيضا لكل مستخدم جديد

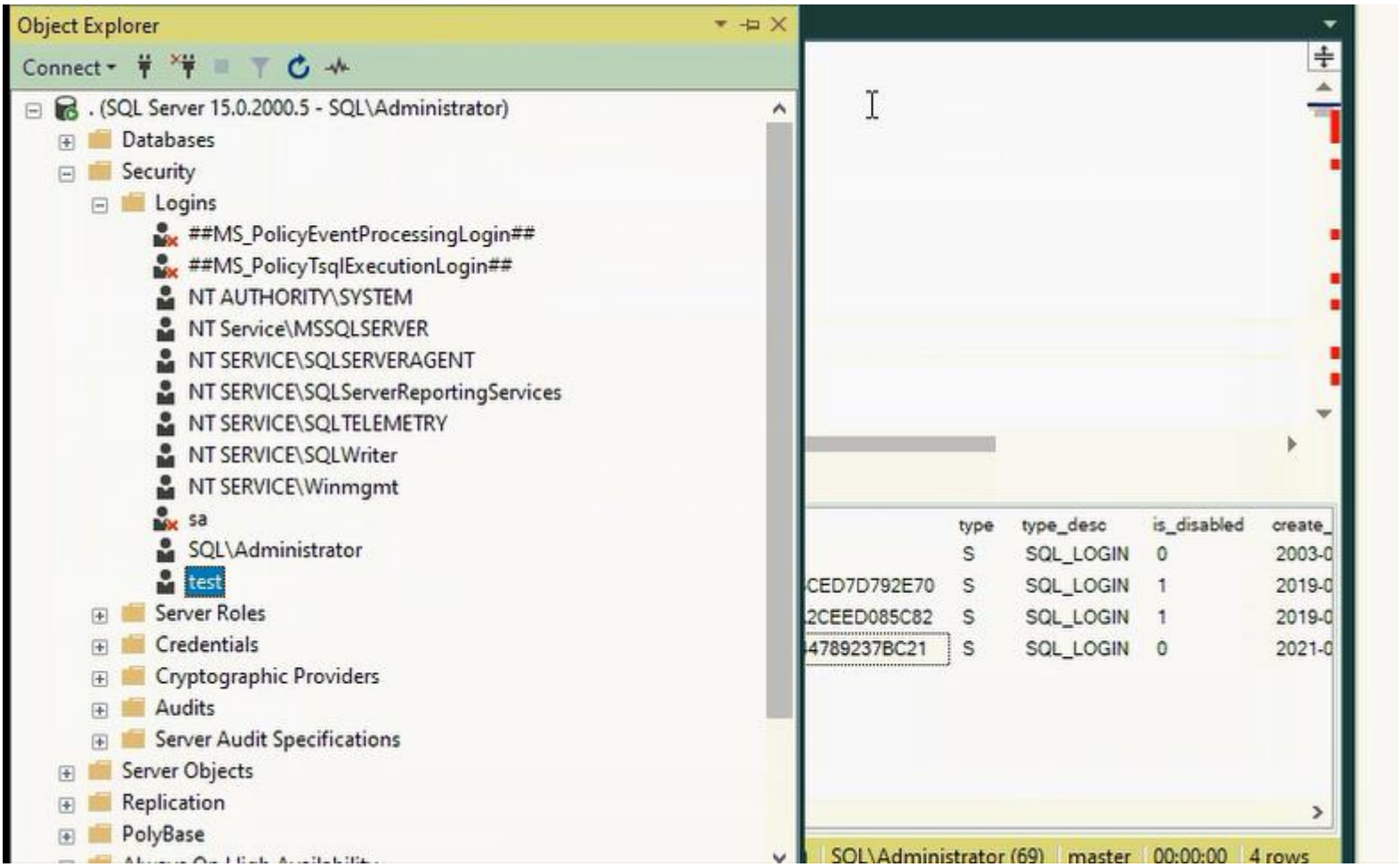
حيث ممكن إنشاء عدة users بنفس الأسماء, و لكن ال SQL Engine سيفرّق بينهم أنه ينظر إلى ال SID الخاص بهم

و لعرضهم: select * from sys.sql_logins

حيث أن لدينا مستخدم sa ( System Administrator ) يكون أول مستخدم منشأ بشكل إفتراضي – و يفضّل عمل Disable له كنوع من الحماية

و قيمة ال principle-id = 1 وال SID = 0x01 ,

لتغيير خواص مستخدم ما, او تعطيل, حذف مستخدم ما , نستطيع من الواجهة الرسومية تعدل خصائصه بعد تحديده:




- للتعديل على طريقة ال Authintication في ال SQL DB, نستطيع التحديد أثناء التنصيب
- أو من Properties الخاصة بال DB خاصتنا و من نافذة Security نحدد النوع ( يوجد نوع هجين مشترك – Mixed Mode يستخدم ال windows, و ال SQL server )
-

The SID assigned to a newly created SQL Server–authenticated login is generated by the SQL Server. This is why two logins with the same names on two SQL Server instances will have differentSIDs (more on this later).

You can use SQL Server Authentication to connect to on-premises SQL Server instances, Azure virtual machine–based SQL Server instances, and databases in Azure SQL Database, but other methods of authentication are preferred in all cases.

SQL Server installs with a SQL Server login named sa (an abbreviation of "system administrator").

Assign a strong password to the sa login and do not use the sa login in your application. The sa login maps to the sysadmin fixed server role, which has irrevocable administrative credentials on the whole server (we will discuss it in more details later in this chapter).

The last three authentication types are exclusive to Azure-based resources, specifically Azure SQL Database or Azure SQL Data Warehouse, using Azure Active Directory (Azure AD) credentials.

**في حال كان لدينا ال SQL server engine موجود على ال cloud نقوم بالمصادقة بإستخدام أحد الخيارات الثلاث الخاصة بال Azure :**
**و هذه الطرق الثلاث ظهرت بعد نسخة ال 2016**

### 2.1.3. Azure Active Directory Universal Authentication

**مثل:**

**Azure Active Directory universal Authentication**


Active Directory - Universal with MFA is an interactive work flow that supports Azure Multi-Factor Authentication (MFA).

تعتمد على طريقتين للمصادقة لتحقيق أمان إضافي

Azure MFA helps safeguard access to data and applications while meeting user demand for a simple sign-in process. It delivers strong authentication with a range of easy verification options-phone call, text message, smart cards with pin, or mobile app notification- allowing users to choose the method

they prefer. When the user account is configured for MFA the interactive authentication work flow requires additional user interaction through pop-up dialogboxes, smart card use, etc. When the user account is configured for MFA, the user must select AzureUniversal Authentication to connect. If the user account does not require MFA, the user can still use the other two Azure Active Directory Authentication options.

This method, like the next two Azure AD–based authentication methods, was first supported by SQL Server Management Studio as of SQL Server 2016.

### 2.1.4. Azure Active Directory Password Authentication

**تعتمد فقط على المصادقة بإستخدام إسم مستخدم و كلمة سر , أثناء المصادقة مع server على ال cloud**

Azure Active Directory Authentication is a mechanism of connecting to MicrosoftAzure SQL Database by using identities in Azure Active Directory (Azure AD). Use this method for connecting to SQL Database if you are logged in to Windows using credentials from a domain that is not federated with Azure, or when using Azure AD authentication using Azure AD based on the initialor the client domain.

Federation is a collection of domains that have established trust. You can federate your on-premises environment with Azure AD and use this federation for authentication and authorization. This sign- in method ensures that all user authentication occurs on-premises.

### 2.1.5. Active Directory Integrated Authentication

**سيتم إستخدام معلومات الحساب الخاص بالمستخدم على ال Azure Active Directory دون الحاجة لإدخال اسم المستخدم و كلمة المرور يدويا ( مثل ال Windows Authentication )**

**حيث تكون integrated مع ال domain المنتمي له**

**أو segrated ( integrated مع domain آخر , بينه و بين ال domain الذي ننتمي له علاقة trust )**

Azure Active Directory Authentication is a mechanism of connecting to Microsoft Azure SQL Database by using identities in Azure Active Directory (Azure AD). Use this method for connecting to SQL Database if you are logged in to Windows using your Azure Active Directory credentials from a federated domain.

You can use Azure AD accounts for authentication very similarly to Windows Authentication, for use when you can sign in to Windows with your Azure AD credentials. No user name or password is requested; instead, your profile's local connections are used.

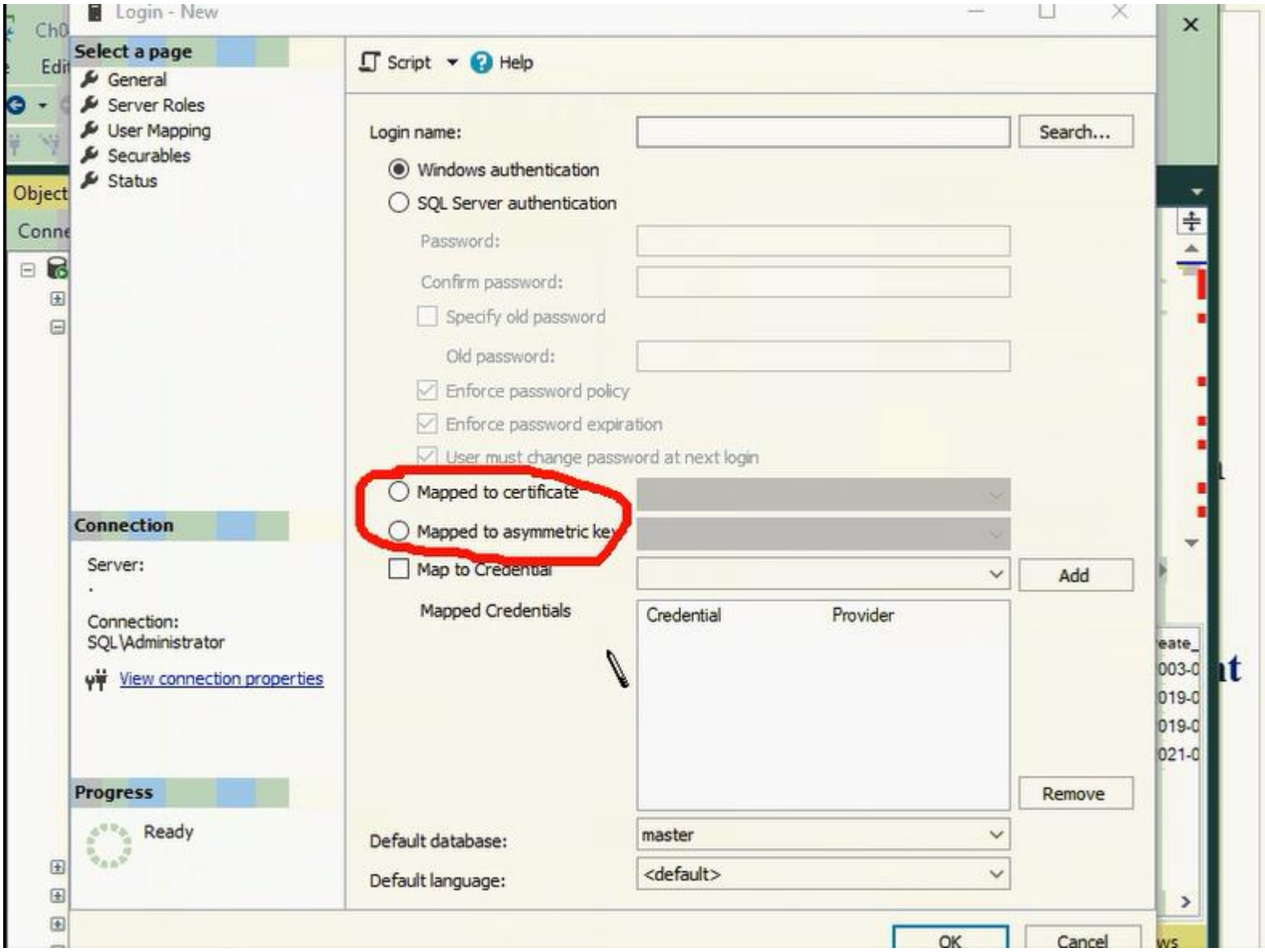
### 2.1.6. Other types of logins

You can create another type of login aside from Windows or SQL Server authentication; however, this type of login has limited uses.

You can create a login that is mapped directly to a certificate

Or mapped to an asymmetric key.

تستعمل هذه الطريقتين عند التعامل مع SQL Server عندما يكون مفعّل عليه ميزات ال Broker المتعلقة بالرسائل, و هي ليست ضمن المنهاج




نستخدم خيار Map to Credential, عندما نريد أن نعطي حساب login معيّن صلاحيات خارج قاعدة البيانات ( على قرص معيّن مثلا ...)

Secure accessto the SQL Server instance is then possible by any client with the public key of the certificate, usinga nondefault endpoint that you create specifically for this type of access.

The Service Broker feature of SQL Server, used for asynchronous messaging and queueing, supports Certificate-Based Authentication, for example.

عند الإتصال بـSQL DB موجودة على نظام ويندوز من نظام لينكس, يتم إستخدام طريقة SQL Server Authentication

أو نستطيع إستخدام طريقة ال Windows Authentication بعد ان نقوم ب join لجهاز اللينكس, على ال Windows Domain
( بإستخدام طريقة ال realm join – عن طريق ال Kerberos )

You also can use these logins to sign database objects, such as stored procedures. This encrypts the database objects and is common for when the code of a third-party application is proprietary and not intended to be accessible by customers.

For example, the ##MS_PolicyEventProcessingLogin## and ##MS_PolicyTsqlExecutionLogin## logins, created automatically with SQL Server, are certificate-based logins.

### *2.2. Authentication to SQL Server on Linux*

You can make SQL Server connections to instances running on the Linux operating system by using Windows Authentication and SQL Authentication.

In the case of SQL Authentication, there are no differences when connecting to a SQL Server instance running on Linux with SQL Server Management Studio.

It is also possible to join the Linux server to the domain (by using the realm join command), using Kerberos, and then connect to the SQL Server instance on Linux just as you would connect to a SQL Server instance on Windows Server.

## *2.3. Factors in securing logins*

In this section, we cover some important topics for logins specifically, including login options, SQL Server configuration, and security governance.

### 2.3.1. Using mixed mode

**عندما توجد علاقة ال trust بين domain1 و domain2 يتم إستخدام ال Kerberos لمصادقة login (user) ( من ال domain1 إلى ال DB في ال domain2 , حيث أن ال windows Authentication هو دائما المفضّل في حال كان متاح**

**نستعمل ال SQL Server Authentication في حال كان ال Kerberos غير متاح بين 2 domains**

Mixed mode is simply a description of a SQL Server that can accept both Windows Authentication and SQL Server Authentication.

As the DBA of a SQL instance, be sure to emphasize to developers and application administrators that Windows Authentication, via named domain accounts or service accounts, is always preferred toSQL Server Authentication.

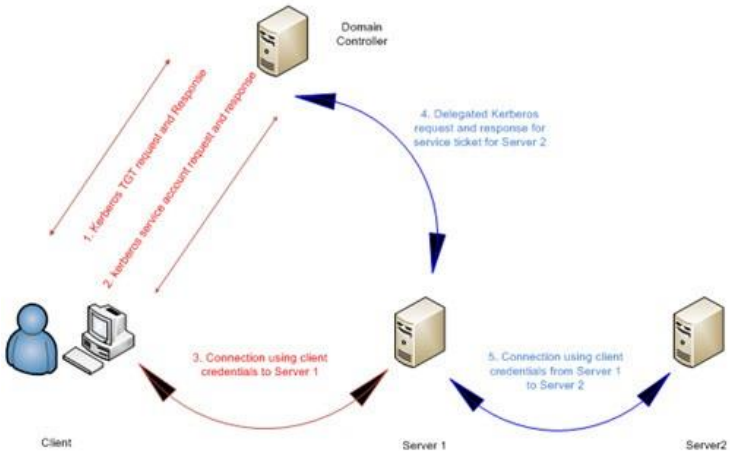
Use SQL Server Authentication to connect to SQL Server instances only in special cases; for example, when Windows-authenticated accounts are impossible to use or for network scenarios involving double-hop authentication when Kerberos is not available.

Since SQL Server 2005, user names and passwords of SQL Server–authenticated logins are no longer transmitted as plain text during the login process. And, unlike early versions of SQL Server, passwords are not stored in plain text in the database. However, any SQL Server–authenticated logincould potentially have its password reverse-engineered by a malicious actor who has access to or a copy of the master database .mdf file. For these reasons and more, Windows-authenticated accounts are far more secure.

**Kerberos & Double hop issues**

Kerberos is a protocol for authenticating service requests between trusted hosts across an untrusted network, such as the internet. Kerberos is built in to all major operating systems, including Microsoft Windows, Apple OS X, FreeBSD and Linux.

Double hop issues are when you have a client connect to one SQL Server and that server needs to pull data from another SQL Server. The first server uses Windows Authentication credentials on the second server and the connection to the first SQL Server is made using Kerberos authentication.




Kerberos Double Hop

ينصح بتفعيل طريقتيني للمصادقة معا : windows authentication, SQL Server Authentication

حيث عند حصول مشكلة بال DC و لا نستطيع المصادقة بإستخدام windows authentication, نستخدم الطريقة الثانية المتاحة و هي ال SQL authentication

### 2.3.2. Login security

In this section, we discuss some important special logins to be aware of, including special administrative access, which you should control tightly.

***The "sa" login***

The "sa" login is a special SQL Server–authenticated login. It is a known member of the sysadmin server role with a unique SID value of 0x01, and you can use it for all administrative access. If your instance is in mixed mode (in which both Windows Authorization and SQL Server Authentication are turned on), DBAs can use the sa account.

The sa account also has utility as the authorization (aka owner) for database objects, schemas, availability groups, and the databases themselves. This known administrator account, however, has

obvious potential consequences.

Applications, application developers, and end users should never use the sa account. This much should be obvious. If it is, consider changing the password and assigning any entities who wereusing it the permissions necessary and appropriate for their role. The sa account, like any SQL

Server–authenticated account, could potentially have its password reverseengineered by a malicious actor who has access to or a copy of the master database .mdf file.

The sa account is a common vector for brute-force attacks to compromise a SQL Server. For this reason, if your SQL Server is exposed to the internet, we recommend that you rename or disable the sa account.

Because its password is commonly known to multiple administrators, it also can serve as an anonymous backdoor for malicious or noncompliant activity by current or former employees. Because your SQL Server instance's DBAs should be using Windows Authentication for all administrative access (more on that later), no DBA should actively use the sa account. Its password doesn't need to be known to DBAs. There are no use cases nor best practices that require accessing the SQL Server by using the sa account.

اسم اي user ( login) له صلاحية sysadmin (role) في أي DB هو : ( يكون اسمه dbo )

حيث ان ال dbo هو DB user له سماحية مطلقة على هذه ال DB ة مرتبط مع كل user له صلاحية sysadmin على هذه ال DB

***-The BUILTIN\Administrators group***

***في نسخة ال 2005 و قبل , عند إنشاء ال SQL DB كان يتم إنشاء حساب بصلاحيات sysadmin لكل ال administrators في الدومين او الجهاز .***

If you have experience in administering SQL Server 2005 or older, you'll remember the BUILTIN\Administrators group, which was created by default to grant access to the sysadmin server role to any account that is also a member of the local Windows Administrator group.

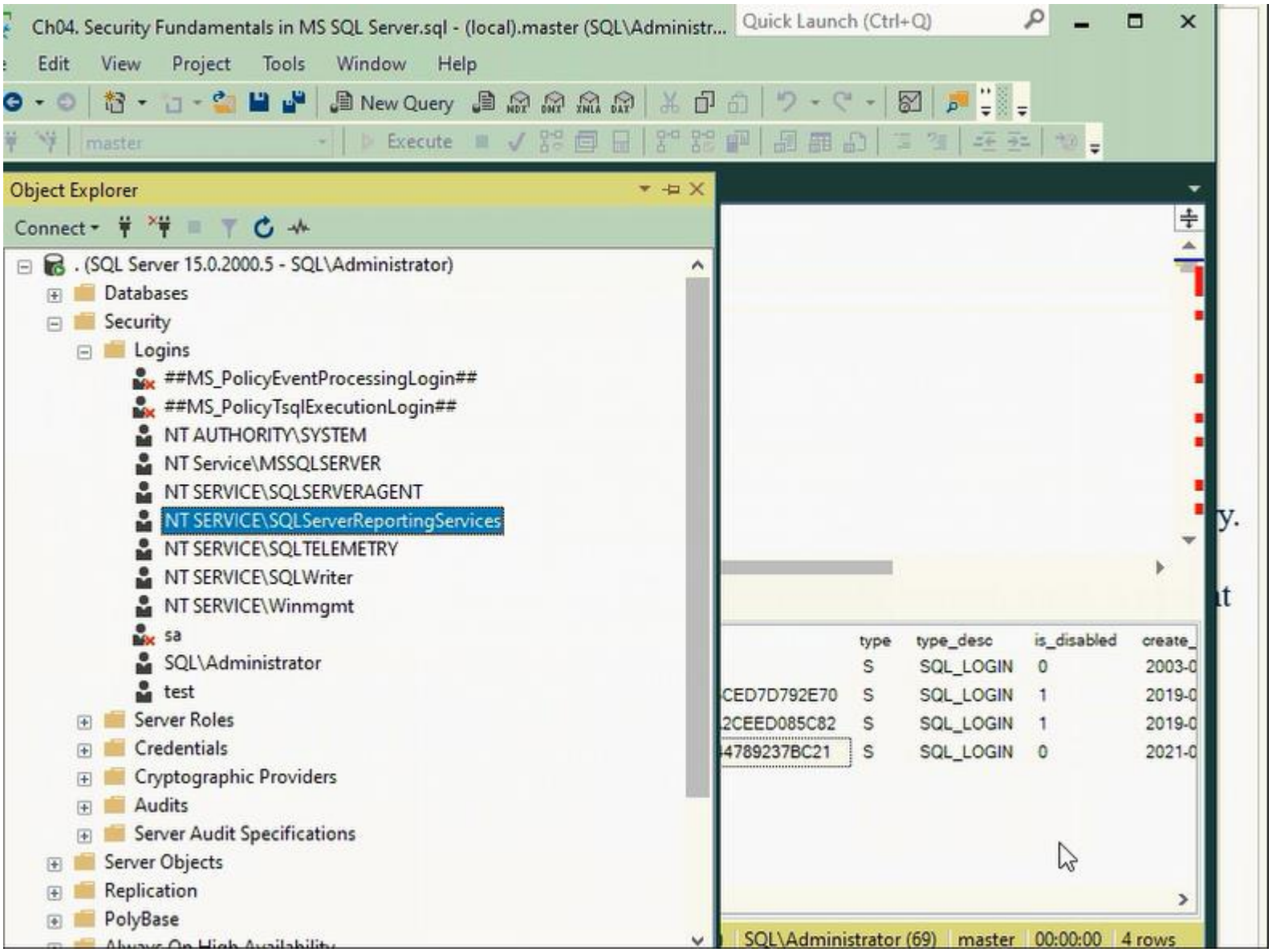
Beginning with SQL Server 2008, this group was no longer added to SQL Server instances by default, because it is an obvious and serious security back door. Although it was potentially convenient for administrators, it was also targeted by malicious actors.

Do not add the BUILTIN\Administrators group to your SQL Server instance—it is no longer there by default for a reason.

***Service accounts***

***ال service account عادة يعطى اقل صلاحيات ممكنة لتشغيل او إدارة خدمة معينة***

***و لاينصح بالتعديل على صلاحيات هذا الحسابات بإستخدام خدمات ويندوز***




Service accounts are usually given the minimal permissions needed to run the services to which they are assigned by SQL Server Configuration Manager. For this reason, do not use the Windows Services (services.msc) to change SQL Server feature service accounts.

Changing the instance's service account with the Windows Services administrative page will likely result in the service's failure to start.

It is not necessary to grant any additional SQL Server permissions to SQL Server service accounts. (You might need to grant additional NT File System (NTFS)–level permissions to file locations, etc.) Although the SQL Server Agent service account likely needs to be a member of the sysadmin role, the SQL Server service account does not. For these reasons and more, different service accounts for different services is necessary.

You also should never grant the NT AUTHORITY\SYSTEM account, which is present by default in a SQL Server instance, any additional permissions. Many Windows applications run under this system account and should not have any nonstandard permissions.

## 3. Managing Principals

Principals are the entities that can request permission to SQL Server resources. They are made up of groups, individuals, or processes. Each principal has its own unique identifier on the server and is scoped at the Windows, server, or database level.

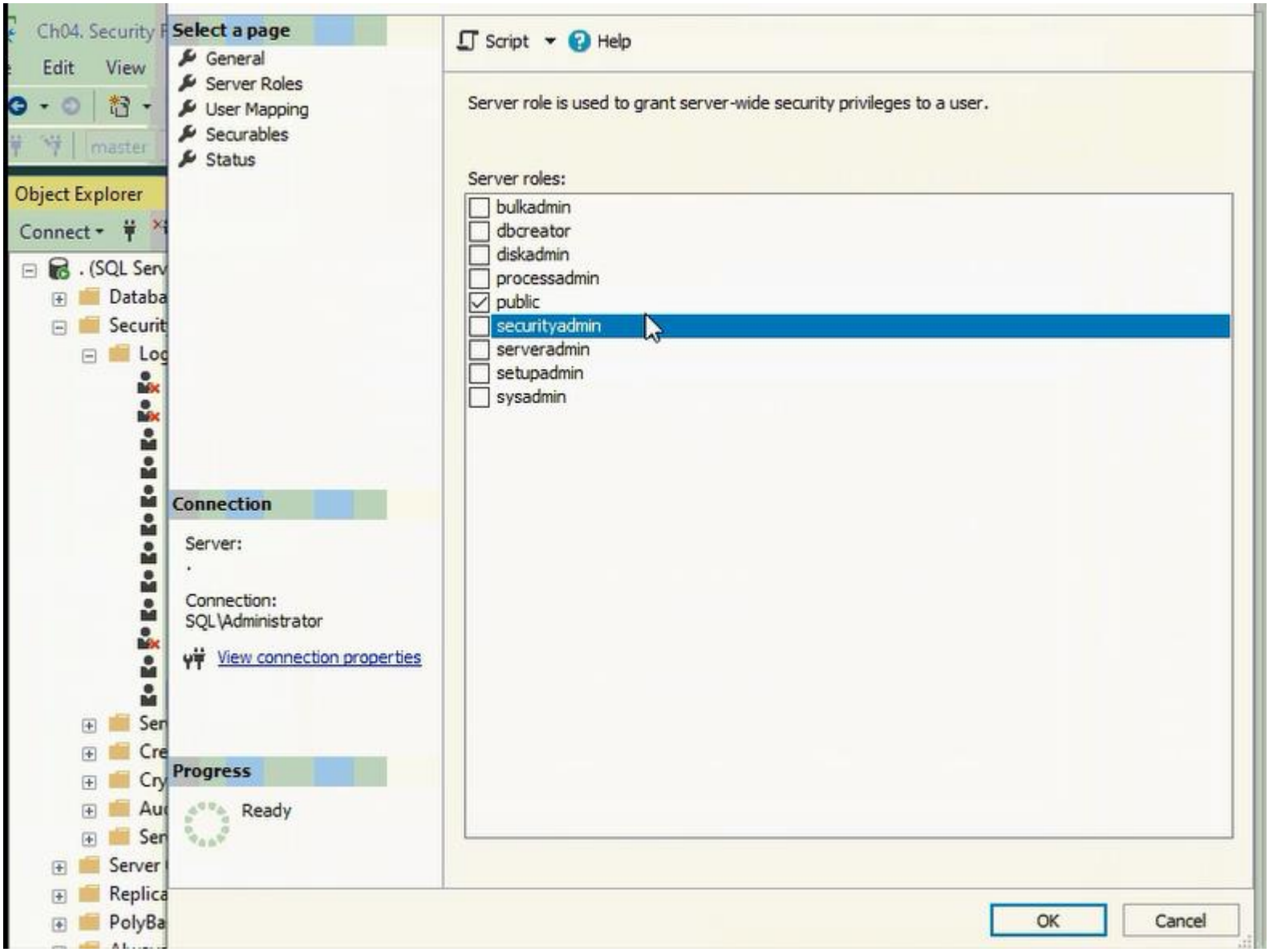
The principals at the Windows level are Windows users or groups. The principals at the SQL Server level include SQL Server logins and server roles. The principals scoped at the database level include database users, data roles, and application roles.

## 4. Role-Based Permissions

**كل Principle يكون له role أو محموعة roles تحدد له صلاحياته**

**حيث أن ال principle (sa) تكون مفعلة لديه ال role الخاصة ب sysadmin, serveradmin و غيرها.. وتسمح له بإستعراض كافة قواعد البيانات في ال SQL Server ( يكون مرتبط مع ال User الإفتراضي dbo الموجود بكل DB )**

- **هناك role إسمها Public تكون مفعّلة لكل ال principles بشكل إفتراضي , حيث لا يمكن إلغاء أي principle منها و تقوم بالسماح لل principle بفقط إستعراض محتوى ال SQL Server من قواعد بيانات و لكن لا يستطيع فتح أي منها و لا تمنحه سماحية عرض أي جدول من أي DB**




Granting permissions to roles rather than to users simplifies security administration. Permission sets that are assigned to roles are inherited by all members of the role.

It is easier to add or remove users from a role than it is to recreate separate permission sets for individual users. Roles can be nested;

حيث نستطيع تعيين عدة permissions داخل role وحدة جديدة ننشأها لسهولة تطبيقها لل users

و هذا في حال أن ال roles الموجودة بالسماحيات التي تحتويها لا تكفي بالغرض

however, too many levels of nesting can degrade performance.

You can also add users to fixed database roles to simplify assigning permissions.

You can grant permissions at the schema level. Users automatically inherit permissions on all new objects created in the schema; you do not need to grant permissions as new objects are created.

- الحالات التي لا نحتاج بها لإنشاء User لل Login:

1) عند ربطه مع ال Sysadmin Role, حيث سيتم ربطه مع ال User الإفتراضي dbo الموجود في كل DB
2) عندما تكون كل وظائفه على ال SQL Server و ليس له أي وظيفة على أي DB
3) عندما ننوي إستخدامه لإنتحال شخصية User آخر بإستخدام تعليمة Impersonate

## *4.1. Server and DB Roles in SQL Server*

يوجد لدينا roles على مستوى ال server و roles على مستوى ال DB

و ليسو متعلقين ببعض, حيث ممكن إضافة roles إلى login على ال server من دون التأثير على صلاحياته لقواعد البيانات

All versions of SQL Server use role-based security, which allows you to assign permissions to a role, or group of users, instead of to individual users. Fixed server and fixed database roles have a fixed set of permissions assigned to them.

### 4.1.1. Fixed Server Roles

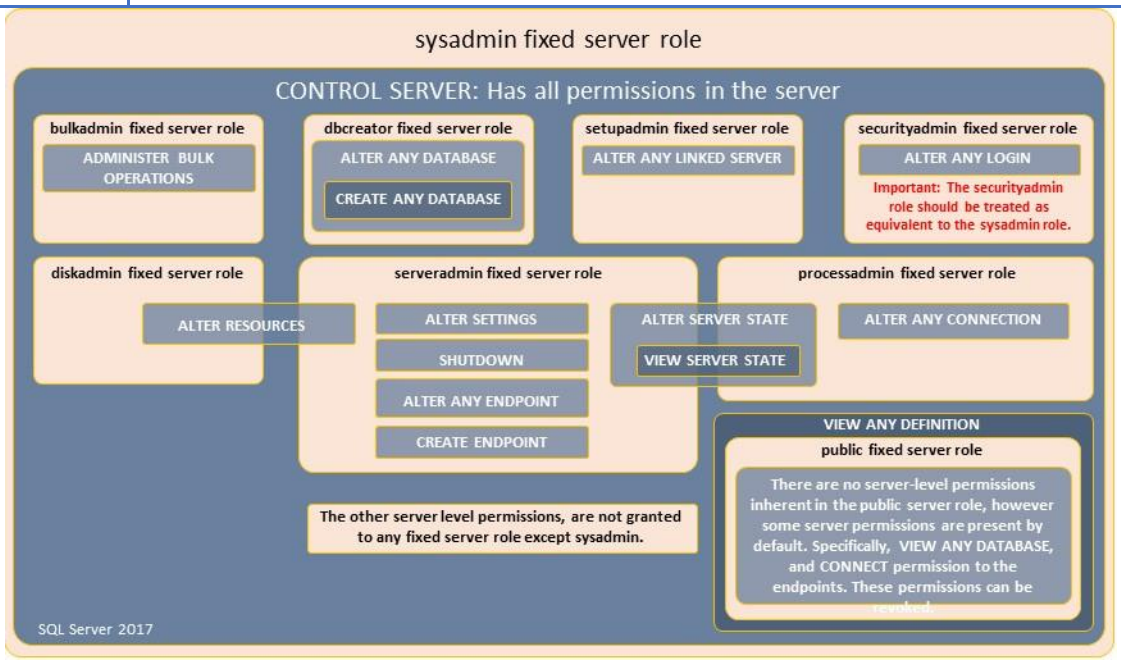
They have a fixed set of permissions and server-wide scope. They are intended for use in administering SQL Server and the permissions assigned to them cannot be changed. Logins can be assigned to fixed server roles without having a user account in a database.

The sysadmin fixed server role encompasses all other roles and has unlimited scope. Do not add principals to this role unless they are highly trusted. Sysadmin role members have irrevocable administrative privileges on all server databases and resources.

| | |
|---|---|
| sysadmin | يستطيع القيام بأي شيئ |
| serveradmin | لهم سماحيات على مستوى ال Configuration و لهم سماحية القيام ب shutdown لل server<br>Members of the serveradmin fixed server role can change server-wide configuration optionsand shut down the server. |
| securityadmin | يستطيع منح او سحب صلاحيات لغيره على مستوى ال server-level , و على مستوى ال database-level<br>و يستطع القيام ب reset لكلمات السر الخاصة بال logins<br>تعتبر role هامة جدا و تعامل بأهمية role ال sysadmin<br>Members of the securityadmin fixed server role manage logins and their properties. Theycan GRANT, DENY, and REVOKE server-level permissions. They can also GRANT, DENY, and REVOKE database-level permissions if they have access to adatabase. Additionally, they can reset passwords for SQL Server logins.<br>IMPORTANT: The ability to grant access to the Database Engine and to configure user permissions allows the security admin to assign most server permissions.<br>The securityadmin role should be treated as equivalent to the sysadmin role. |
| processadmin | يستطيع إنهاء عمليات جارية حاليا في ال SQL Server<br>Members of the processadmin fixed server role can end processes that are running in aninstance of SQL Server. |
| setupadmin | يستطيع إضافة او حذف linked-server<br>Members of the setupadmin fixed server role can add and remove linked servers by using Transact-SQL statements. (Sysadmin membership is needed when using Management Studio.) |

| | |
|---|---|
| bulkadmin | يستطيع أن يقوم ب Bulk-insert لأي table<br>Members of the bulkadmin fixed server role can run the BULK INSERT statement. |
| diskadmin | يستطيع إدارة ال Physical files و الأقراص<br>The diskadmin fixed server role is used for managing disk files. |

| | |
|---|---|
| dbcreator | يستطيع إنشاء, تعديل, حذف,إسترجاع لأي DB<br><br>Members of the dbcreator fixed server role can create, alter, drop, and restore any database. |
| Public | تكون إفتراضية للكل<br><br>Every SQL Server login belongs to the public server role. When a server principal has not been granted or denied specific permissions on a securable object, the user inherits the permissions granted to public on that object. Only assign public permissions on any object when you want the object to be available to all users. You cannot change membership in public. |

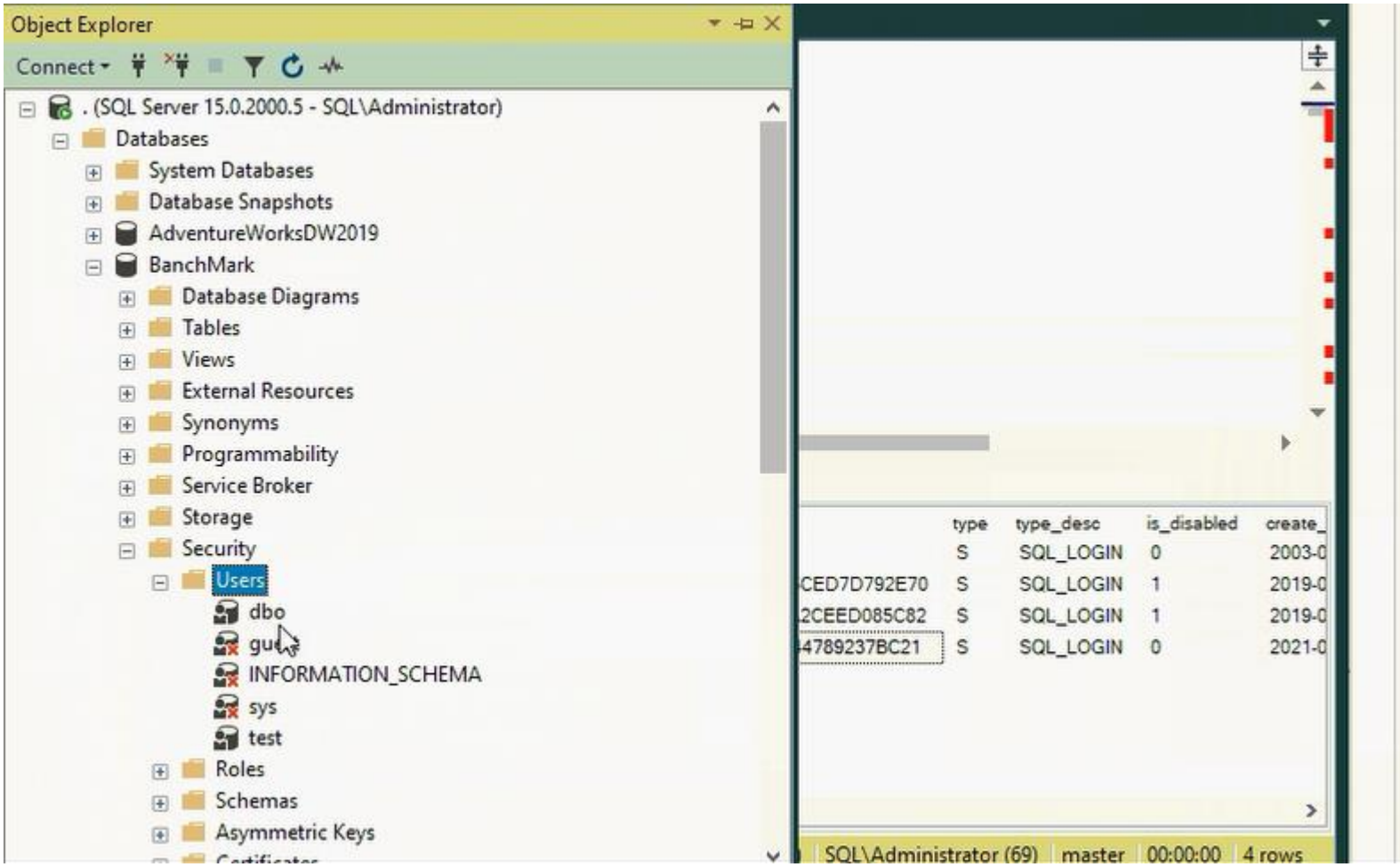



### 4.1.2. Fixed Database Roles in SQL Server

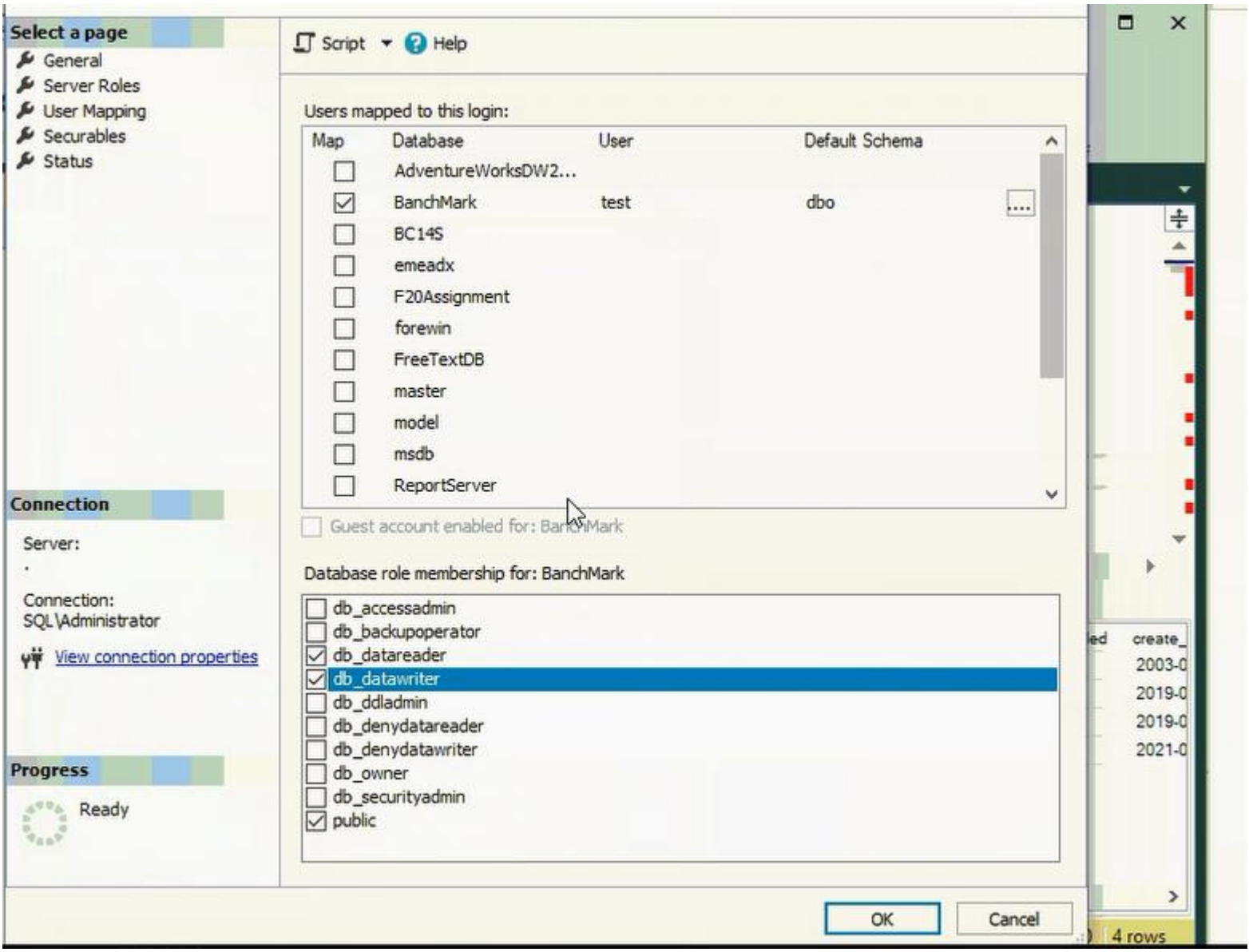
Fixed database roles have a pre-defined set of permissions that are designed to allow you to easily manage groups of permissions.

نستطيع تحديد Role خاصة ب principle معيّن على DB معيّنة فقط, عوضا عن تفعيل ال role على جميع ال DBs

حيث من تبويبة Security الخاصة ب DB معيّنة نحدد ال Principle المراد تعديل ال roles الخاصة به :



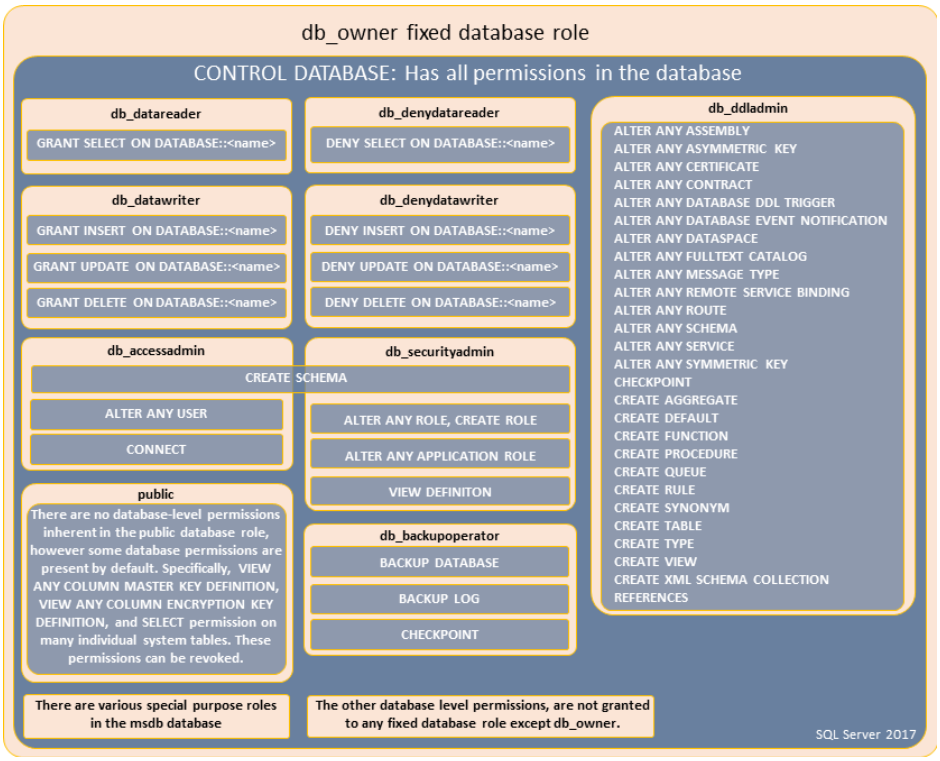

ثم من ال User Mapping نختار الصلاحيات المرادة على هذه ال DB:




| Role name | Description |
|---|---|
| db_owner | له سماحيات كاملة على ال DB, و لكن ليس كسماحيات ال login الذي اسمه dbo<br>حيث أن بعض المهام مثل مهام ال Maintenance ستحتاج من ال Login ان يكون لديه سماحيات على مستوى ال server<br>ايضا , لذلك تعتبر ال dbo أقوى من ال db_owner<br>Members of the db_owner fixed database role can perform all configuration and maintenance activities on the database, and can also drop the database in SQL Server. (In SQL Database and SQL Data Warehouse, some maintenance activities require server-level permissions and cannot be performed by db_owners.) |
| db_securityadmin | يستطيع تعديل سماحيات Login معيّن |

| | |
|---|---|
| | Members of the db_securityadmin fixed database role can modify role membership and manage permissions. Adding principals to this role could enable unintended privilege |

| | |
|---|---|
| | escalation. |
| db_accessadmin | يستطيع إضافة أو إزالة سماحية وصول على ال DB لل sql server , Windows groups , Windows logins logins<br><br>Members of the db_accessadmin fixed database role can add or remove access to the database for Windows logins, Windows groups, and SQL Server logins. |
| db_backupoperator | يستطيع القيام ب backup لل DB<br>Members of the db_backupoperator fixed database role can back up the database. |
| db_ddladmin | يستطيع تنفيذ أي تعليمة DDL في ال DB<br>Members of the db_ddladmin fixed database role can run any Data Definition Language (DDL) command in a database. |
| db_datawriter | يستطيع حذف, إضافة, تعديل بيانات في أي جدول<br><br>Members of the db_datawriter fixed database role can add, delete, or change data in all user tables. |
| db_datareader | يستطيع قراءة محتوى بيانات أي جدول<br>Members of the db_datareader fixed database role can read all data from all user tables. |
| db_denydatawriter | لا يستطيعون التعديل على اي من محتوى ال DB<br>Members of the db_denydatawriter fixed database role cannot add, modify, or delete any data in the user tables within a database. |
| db_denydatareader | لا يستطيعون قراءة أي من محتوى ال DB<br>Members of the db_denydatareader fixed database role cannot read any data in the user tables within a database. |


db_owner fixed database role
CONTROL DATABASE: Has all permissions in the database
db_datareader
GRANT SELECT ON DATABASE::<name>
db_denydatareader
DENY SELECT ON DATABASE::<name>
db_ddladmin
ALTER ANY ASSEMBLY
ALTER ANY ASYMMETRIC KEY
ALTER ANY CERTIFICATE
ALTER ANY CONTRACT
ALTER ANY DATABASE DDL TRIGGER
ALTER ANY DATABASE EVENT NOTIFICATION
ALTER ANY DATASPACE
ALTER ANY FULLTEXT CATALOG
ALTER ANY MESSAGE TYPE
ALTER ANY REMOTE SERVICE BINDING
ALTER ANY ROUTE
ALTER ANY SCHEMA
ALTER ANY SERVICE
ALTER ANY SYMMETRIC KEY
CHECKPOINT
CREATE AGGREGATE
CREATE DEFAULT
CREATE FUNCTION
CREATE PROCEDURE
CREATE QUEUE
CREATE RULE
CREATE SYNONYM
CREATE TABLE
CREATE TYPE
CREATE VIEW
CREATE XML SCHEMA COLLECTION
REFERENCES
db_datawriter
GRANT INSERT ON DATABASE::<name>
GRANT UPDATE ON DATABASE::<name>
GRANT DELETE ON DATABASE::<name>
db_denydatawriter
DENY INSERT ON DATABASE::<name>
DENY UPDATE ON DATABASE::<name>
DENY DELETE ON DATABASE::<name>
db_accessadmin
CREATE SCHEMA
ALTER ANY USER
CONNECT
db_securityadmin
ALTER ANY ROLE, CREATE ROLE
ALTER ANY APPLICATION ROLE
VIEW DEFINITON
public
There are no database-level permissions inherent in the public database role, however some database permissions are present by default. Specifically, VIEW ANY COLUMN MASTER KEY DEFINITION, VIEW ANY COLUMN ENCRYPTION KEY DEFINITION, and SELECT permission on many individual system tables. These permissions can be revoked.
db_backupoperator
BACKUP DATABASE
BACKUP LOG
CHECKPOINT
There are various special purpose roles in the msdb database
The other database level permissions, are not granted to any fixed database role except db_owner.
SQL Server 2017

### 4.1.3. Database Roles and Users

Logins must be mapped to database user accounts in order to work with database objects.Database users can then be added to database roles, inheriting any permission sets associated with those roles. All permissions can be granted.

You must also consider the public role, the dbo user account, and the guest account when you design security for your application.

***The public Role***

لا نستطيع سحب ال ***public*** ***role*** من أي ***use*** (***principle***)
و نستطيع أن نضيف عليها سماحيات

Contained in every database including system databases. It cannot be dropped and you cannot add or remove users from it. Permissions granted to the public role are inherited by all other users and roles because they belong to the public role by default. Grant public only the permissions you want all users to have.

***The dbo User Account***

اسم اي user (login ) له صلاحية sysadmin (role) في أي DB هو : ( يكون اسمه dbo )

حيث ان ال dbo هو DB user له سماحية مطلقة على هذه ال DB ة مرتبط مع كل user له صلاحية sysadmin على هذه ال DB

The dbo, or database owner, is a user account that has implied permissions to perform all activities in the database. Members of the sysadmin fixed server role are automatically mapped to dbo.

dbo is also the name of a schema. The dbo user account is frequently confused with the db_owner fixed database role. The scope of db_owner is a database; the scope of sysadmin is the whole server.

Membership in the db_owner role does not confer dbo user privileges.

***The guest User Account***

***هو حساب له سماحية دخول إلى ال SQL Server و لكن ليس لديه أي login ( user ) على أي DB لذلك يتم إعطاءه صلاحيات ال guest في حال كان login Enabled***

***و يكون افتراضيا Disabled و ينصح إبقائها على هذا الحال***

***لعمل Disable لل Guest user عن طريق تعليمة:***

***Revoke connect from guest***

***و في حال كان لديه login ( user) على ال DB سيأخذ صلاحيات ال Public الإفتراضية***

After a user has been authenticated and allowed to log in to an instance of SQL Server, a separate user account must exist in each database the user has to access. Requiring a user account in each database prevents users from connecting to an instance of SQL Server and accessing all the databases on a server.

The existence of a guest user account in the database circumvents this requirement by allowing a login without a database user account to access a database. The guest account is a built-in account in all versions of SQL Server.

By default, it is disabled in new databases. If it is enabled, you can disable it by revoking its CONNECT permission by executing the Transact-SQL REVOKE CONNECT FROM GUEST statement.

## *4.2. Solving orphaned SIDs*

***لدينا مثال لنتعرف على ال Orphaned SID:***

```
Drop database if exists svu1

Create database svu1 on
(name='svu_data1' , filename='E:\svu\bait\....\svu_data1.mdf')
Log on (name='svu1_log' , filename='E:\svu\bait\...\svu1_log.ldf')

Use svu1

Create table test ( id int )

Go

--------drop login  [ Orphanlogin ]

Use [ master ]
```

***ملاحظة لإستخدام تعليمة Use db-name:***
***فيك تستخدم الحالتين: use db-name أو use [db_name]***
***الحالة التانية بتجي أفضل لسبب اذا كان عندك داتابيس بإسما في فراغات مثلا او Domain user***
***لأنو الدومين يوزر بيجي هيك Domain\user***
***وال Sql server بس يشوف فراغات او | مابيقرا كلشي كمصطلح واحد .. لذلك بتحتاج تحط []***

```
Go
```

***نقوم بإنشاء login جديد:***

```
Create login [orphanlogin] with password=n'123' , check_policy = off
```

***إستخدمناها ليتم السماح بإستخدام كلمة سر ضعيفة مثل المستخدمة***

```
Go
Use [ svu1 ]
Go
```

***نقوم بإنشاء user لل login الذي أنشأناه سابقا:***

```
Create user [orphanlogin] for login [orphanlogin]
Go
```

***نقوم بإعطاء هذا ال User صلاحيات قراءة و كتابة عبر إضافته لل Role الخاصة بكل منهما:***

```
Alter role [db-datareader] add member [orphanlogin]
Go
Alter role [db-datawriter] add member [orphanlogin]
Go
```

نقوم بالتأكد عبر عرض جميع ال Logins:

```
Select * from sys.sql.logins where name='orphanlogin'

Backup database svu1 to disk 'E:\svu\bai\...\svu1backup.bak'

Use master
Drop database svu1
```

----------on another server:

هذا السيناريو يحدث عند مثلا تغيير ال Server الذي نعمل عليه, أو نقل ال DB إلى غير شخص على غير جهاز ليكمل العمل عليها مثلا ...

نقوم بتنفيذ هذه التعليمات بعد الإتصال ب Server غير المتصلين به حاليا في ال SQL Server management Studio:

نقوم بالتأكد من عدم وجود login لدينا بنفس الإسم, ثم نقوم ب restore لل DB:

```
Drop login [orphanlogin]

Restore database [svu1] from disk='E:\svu\bait\...\svu1backup.bak'
with move N'svu1_data' to N'C:\program files\microsoft sql sever\svu1_data'
move N'svu1_log' to N'C:\ program files\microsoft sql sever\svu1_log'

use svu1
```

نقوم بالتأكد من وجود ال user الذي إسمه orphanlogin بعد إسترجاع ال DB على ال Server الجديد:

```
select * from sys.sysusers
```

-------try to login as orphanlogin,
-------failed.., create the login

```
Use [master]
Go
```

ملاحظة للتذكرة: ال Users يتم حفظهم على ال DB, بينما ال Logins يتم حفظهم على ال master

```
Create login [orphanlogin] with password=N'123' , check_policy=off
```

----------try to login and read data from svu1

لن نستطيع قراءة أي معلومات من ال svu1

-------why?

بسبب أن ال user و ال login الذين بإسم orphanlogin ليس لهم نفس ال SID
حيث أن ال user تم إسترجاعه مع ال DB, و ال Login تم إنشائه يدويا على السيرفر الجديد

***و هذه الحالة تسمّى ب Orphaned SID حيث لا يكون ال user مربوط بال login لإختلاف ال SID***
***و تحدث مثلا عند إستخدام نفس إسم ال username بعد نقل السيرفر لغير حاسب, أو عند تغيير الدومين و إستخدام نفس ال Username, أو نسخ ال DB إلى غير سيرفر .....***
***و للتأكد نقوم بعرض ال logins و ال users لدينا, و نرى إختلاف ال SID بين ال login و ال user المذكورين:***

```
Select * from sys.sql.logins where name='orphanlogin'
Use svu1
Select * from sys.sysusers where name='orphanlogin'
```

***----------the solution:***

***و الحل يكون بتعديل ال user ليرتبط مع ال login:***

```
Use svu1
Alter user orphanlogin with login =orphanlogin
```

***و للتأكد:***

```
Select * from sys.sql.logins where name='orphanlogin'
Use svu1
Select * from sys.sysusers where name='orphanlogin'
```

***سنرى توافق إستخدام نفس ال SID الآن, و قدرتنا على قراءة المعلومات و الدخول إلى ال DB بإستخدام ال login نفسه***

***و لتفادي هذه المشكلة نستطيع عند إنشاء ال login على ال server الجديد ليطابق نفس إسم ال user, تحديد إستخدام SID مطابق لل SID الخاص بال user:***

```
Create login orphanlogin with password=N'123' , check-policy=off,  sid='…….'
```

An orphaned SID is a user who is no longer associated with its intended login. The user's SID no longer matches, even if the User Name and Login Name do.

Any time you restore a (noncontained) database from one SQL Server to another, the database users transfer, but the server logins in the master database do not. This will cause SQL Server– authenticated logins to break, but Windows-authenticated logins will still work.

***Why?***

The key to this common problem that many SQL DBAs face early in their careers is the SID and, particularly, how the SID is generated.

For Windows-authenticated logins based on local Windows accounts and SQL Server–authenticated logins, this SID will differ from server to server.

The SID for the login is associated with the user in each database based on the matching SID. The names of the login and user do not need to match, but they almost certainly will, unless you are a SQL DBA intent on confusing your successors. But know that it is the SID, assigned to the login andthen applied to the user, that creates the association.

The most common problem scenario is as follows:

1. A database is restored from one SQL Server instance to another.
2. The SIDs for the Windows-authenticated logins and their associated users in the restored database will not be different, so Windows-authenticated logins will continue to authenticate successfully and grant data access to end users via the database users in each database.
   The SIDs for any SQL Server–authenticated logins will be different on each server. So, therefore, will the associated database users' SIDs in each database. When restoring the database from one server to another, the SIDs for the server logins and database users no longer match. Their names will still match, but data access cannot be granted to end users.

3. The SID must now be rematched before SQL Server–authenticated logins will be allowed access to the restore database.

***Problem scenario***

A database exists on Server1 but does not exist on Server2.

**Original state**

Server1

SQL Login = Katherine

SID = 0x5931F5B9C157464EA244B9D381DC5CCC

Database User = Katherine

SID = 0x5931F5B9C157464EA244B9D381DC5CCC

Server2

SQL Login = Katherine

SID = 0x08BE0F16AFA7A24DA6473C99E1DAADDC

Then, the database is restored from Server1 to Server2. Now, we find ourselves in this problem scenario:

**Orphaned SID**

Server1

SQL Login = Katherine

SID = 0x5931F5B9C157464EA244B9D381DC5CCC

Database User = Katherine

SID = 0x5931F5B9C157464EA244B9D381DC5CCC

Server2

SQL Login = Katherine

SID = 0x08BE0F16AFA7A24DA6473C99E1DAADDC

Database User = Katherine

SID = 0x5931F5B9C157464EA244B9D381DC5CCC ← Orphaned SID

***The resolution***

The resolution for the preceding example issue is quite simple:

```
ALTER USER Katherine WITH LOGIN = Katherine;
```

The SID of the user is now changed to match the SID of the login on Server2; in this case,

0x08BE0F16AFA7A24DA6473C99E1DAADDC. Again, the relationship between the Server Login and the Database User has nothing to do with the name.

You can use the script that follows to look for orphaned SIDs. This should be a staple of your DBA toolbox. It's important to understand that this script only guesses that logins and user names should have the same name. If you are aware of logins and users that do not have the same name and should be matched on SID, you will need to check for them manually.

```
Select
DBUser_Name = dp.name
, DBUser_SID = dp.sid
, Login_Name = sp.name
, Login_SID = sp.sid
, SQLtext = 'ALTER USER [' + dp.name + ']
WITH LOGIN = [' + ISNULL(sp.name, '???') + ']'
from sys.database_principals dp
left outer join sys.server_principals sp
on dp.name = sp.name
where
dp.is_fixed_role = 0
and sp.sid <> dp.sid
and dp.principal_id > 1
and dp.sid <> 0x0
order by dp.name;
```

You should check for orphaned SIDs every time you finish a restore that brings a database from one server to another; for example, when refreshing a preproduction environment.

***Preventing orphaned SIDs***

Orphaned SIDs are preventable. You can re-create SQL Server–authenticated logins on multiple servers, each having the same SID. This is not possible using SQL Server Management Studio; instead, you must accomplish this by using the CREATE LOGIN command, as shown here:

```
CREATE LOGIN [Katherine] WITH PASSWORD=N'strongpassword', SID =
0x5931F5B9C157464EA244B9D381DC5CCC;
```

Using the SID option, you can manually create a SQL login with a known SID so that your SQL Server–authenticated logins on multiple servers will share the same SID. Obviously, the SID must be unique on each instance.

In the previous code example, we used sys.server_principals to identify orphaned SIDs. You can also use sys.server_principals to identify the SID for any SQL Server– authenticated login.

Creating SQL Server–authenticated logins with a known SID is not only helpful to prevent orphaned SIDs, but it could be crucially timesaving for migrations involving large numbers of databases, each with many users linked to SQL Server–authenticated logins, without unnecessary outage or administrative effort.

## *4.3. Ownership versus authorization*

- ال ***Ownership*** هو الشخص الذي لديه صلاحية كاملة على ال ***DB***
عند إنشاء ***DB*** من مستخدم, هذا ال ***user*** يكون ال ***Owner*** بدون أن يوجد إسمه ضمن ال ***DB-Owners***
و لكن سيصبح موجود في ال ***owner-sid*** في ال ***sys.databases***
و قد تم تعديل إسم مصطلح ال ***Ownership*** من بعد نسخة ال 2005 ليصبح المصطلح إسمه ***Authorization***

مثال عملي للتأكد من عدم وجود ال ***User*** الذي ينشأ ال ***DB*** ضمن ال ***db-owners*** :

```
Drop svu1 if exists

Create database svu1 on
  (name='svu_data1' , filename='E:\svu\bait\....\svu_data1.mdf')
  Log on (name='svu1_log' , filename='E:\svu\bait\...\svu1_log.ldf')

Go
```

***-----view owners:***

نقوم بعرض ال ***Owners*** مع ال ***login*** الخاص بهم:

```
Select d.name, s.name as [owner], d.owner-sid from sys.databases d
Inner join sys.syslogins s on d.owner_sid=s.sid
```

يتم الوقوع بمشاكل خاصة بال ***impersonation*** ( إنتحال الشخصية)عند تعطيل ال ***user*** الخاص بال ***sid*** الخاص بال ***owner*** أو تغيير حساب ال ***Owner*** في ال ***active directory***
حيث لن نستطيع التعديل على أي من ال ***child objects*** الخاصة بال ***DB*** ( بسبب عدم وجود صلاحية حساب ال ***owner*** - ليس ***Valid*** )
أو بنقل ال ***DB*** في حال إردنا ذلك ...

***---------change owner:***

ملاحظات مهمة:

- ال ***Owner*** ليس شرط أن يكون له ***User*** داخل ال ***DB*** ( مجرد ***login*** يكفي )
- ليس إجباري أن يكون لكل ***User*** - ***login*** خاص به, ولكن كل ***login*** إجباري أن يكون له ***User***

لتغيير ال ***Onwership***:

في النسخ القديمة نستخدم ال ***stored-procedure*** التالية:

***Sp_changedbowner***

بينما في النسخ الحديثة ( حين أصبح مسمّى المصطلح هو ***Authorization*** ):

***Alter authorization on database :: [db_name] to [server principle];***

الأفضل و ال ***Recommended*** يكون ال ***Owner*** هو ال ***sa***, أو حساب بصلاحيات ال ***sa*** بعد ان نكون قد غيرنا الإسم الإفتراضي لحساب ال ***sa*** لأسباب أمنية ( تغيير كل الحسابات ال ***Default*** )

***Alter authorization on database :: svu1 to sa***

Database ownership is an important topic and a common check-up finding for SQL Server administrators. The concept of an ownership chain is complicated to explain; in this section, we cover the topic of database ownership and its impact.

Beginning with SQL Server 2008, “ownership” was redefined as “authorization.” Ownership is now a casual term, whereas AUTHORIZATION is the concept that establishes this relationship between

an object and a principal.

Changing the AUTHORIZATION for any object, including a database, is the preferred, unified terminology than describing and maintaining object ownership with a variety of syntax and management objects. In the case of a database, however, although AUTHORIZATION does not

imply membership in the db_owner role, it does grant the equivalent highest level of permissions. For this reason, and for reasons involving the database ownership chain, named individual accounts (for example, your own [domain*firstname.lastname*]) should not be the AUTHORIZATION of a database.

The problem—which many developers and administrators do not realize—is that when a user creates a database, that user is the "owner" of the database, and that user principal's SID is listed as the owner_sid in sys.databases.

If the database's "owner_sid" principal account was ever to be turned off or removed in Active Directory, and you move the database to another server without that principal, you will encounter problems with IMPERSONATION and AUTHORIZATION of child objects, which could surface as a wide variety of errors or application failures. This is because the owner_sid is the account used as the root for authorization for the database. It must exist and be a valid principal.

For this reason, DBAs should change the AUTHORIZATION of databases to either a known high-level, noninteractive service account or to the built-in sa principal (sid 0x01). It is a standard item on any good SQL Server health check.

If there are databases with sensitive data that should not allow any access from other databases, they should not have the same owner_sid as less-secure databases, and you should not turn on Cross Database Ownership Chaining at the server level (it is not by default).

***Changing database ownership***

When "ownership" was redefined as "authorization," the stored procedure sp_changedbowner was deprecated in favor of the ALTER AUTHORIZATION syntax; for example:

```
ALTER AUTHORIZATION ON DATABASE::[databasename] TO [server_principal];
```

لكي يستطيع أي user أن يغير ال Ownership الخاصة بأي DB, يجب تحقق الشروط التالية:

في حال ال SQL Server من نوع on-premises:

- أن يكون ال user من نوع sysadmin, أو يمتلك صلاحية Take Ownership على هذه ال DB
- أن يكون لديه صلاحية Impersonate على حساب ال Owner الجديد

( في حال لدى ال User صلاحيات sysadmin فلا يحتاج للحصول على صلاحية impersonate )

- يجب على حساب ال Owner الجديد أن لا يكون موجود ضمن الحسابات ( Users ) الموجودة في ال DB هذه, و في حال وجوده فيجب حذفه

بإستخدام تعليمة drop قبل إستخدام تعليمة ال alter authorization

و في حال ال SQL Server من نوع Azure:

- يجب أن يكون حساب ال Owner الجديد من نوع Authenticated-login بإستخدام ال AD

أو أن يكون من نوع Federated أو Managed , يعني يتصل من غير domain و هناك علاقة Trust بين الدومينز المختلفة

- ليس ضروري على الحساب الذي يقوم بعملية تغيير ال Ownership أن يكون من نوع sysadmin بسبب عدم وجود هذا النوع في ال Azure
- يجب على الحساب الذي يقوم بتغيير ال Ownership أن يكون ال Owner الحالي لل DB

أو ان يكون حساب ال administrator المنشأ بشكل إفتراضي في ال DB عند إنشائها

أو أن يكون حساب administrator من حسابات ال admins الخاصة بال AD

حيث نصل لملاحظة مهمة بالنسبة لتعديل ال Ownership في ال Azure:

فقط حسابات ال Azure AD تستطيع التعديل على حسابات ال Azure AD

أما حسابات ال sql-server authenticated تستطيع إدارتها أو تعديل حساباتها بإستخدام ال sql-server authenticated أو حساب Azure AD

In SQL Server databases, the new owner can be a SQL Server–authenticated login or a Windows-authenticated login. To change the ownership of a database by running the ALTER AUTHORIZATION statement, the principal that's running needs the TAKE OWNERSHIP permission and the IMPERSONATE permission for the new owner.

The new owner of the database must not already exist as a user in the database. If it does, the ALTER will fail with the error message: "The proposed new database owner is already a user or

aliased in the database." You will need to drop the user before you can run the ALTER AUTHORIZATION statement.

For Azure SQL Database, the new owner can be a SQL Server–authenticated login or a user object federated or managed in Azure AD, though groups are not supported.

To change the ownership of a database in Azure SQL Database, there is no sysadmin role of which to be a member. The principal that alters the owner must either be the current database owner, the administrator account specified upon creation, or the Azure AD account associated as the administrator of the database. As with any permission in Azure SQL Database, only Azure AD accounts can manage other Azure AD accounts. You can manage SQL Server–authenticated accounts by SQL Server–authenticated or Azure AD accounts.

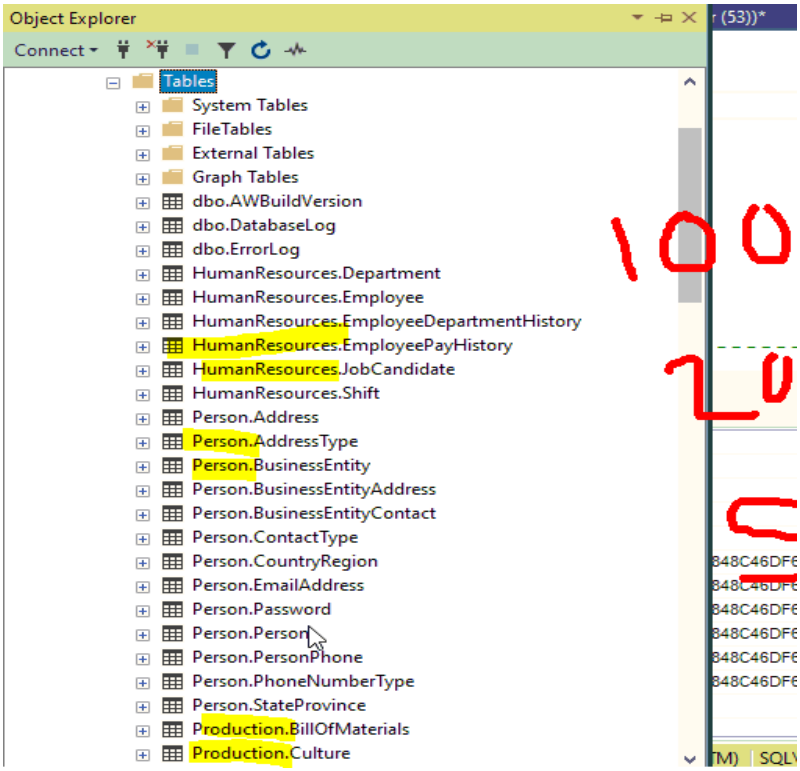
## 5. Ownership & User-Schema Separation

A core concept of SQL Server security is that owners of objects have irrevocable permissions to administer them. You cannot remove privileges from an object owner, and you cannot drop users from a database if they own objects in it.

- نذكر سابقا, لتفعيل عدة سماحيات معيّنة لمجموعة من ال users, عوضا عن تطبيق كل سماحية بشكل منفرد لكل user

نقوم بجمع هذه السماحيات جميعا ضمن Role و نطبّق هذه ال Role على كل User فتصبح العملية أسرع و أسهل

- ال Schema هي Logical Container لل Objects تفيد بأمور تنظيمية و أمنية




حيث تفيد في الأمور التنظيمية بسبب وجودها في بداية إسم كل جدول فنستطيع تمييز هذه الجداول تنتمي لأي Schema, مثل:

## 5.1. User-Schema Separation

حيث في النسخ القديمة من ال SQL-Server كان ال user مرتبط بشكل مباشر مع ال Schema, حيث كان ال user عند إنشاء جدول جديد يصبح إسم الجدول: user.table-name عوضا عن الوضع الحالي schema.table-name

بينما في النسخ الحديثة قامت ميكروسوفت بفصل ال schema عن ال User, حيث كل user أصبح يستطيع الحصول على صلاحيات مختلفة على كل schema , حيث أصبح ممكن إمتلاك Owner معيّن ل Schema بدون أن تسمّى بإسمه

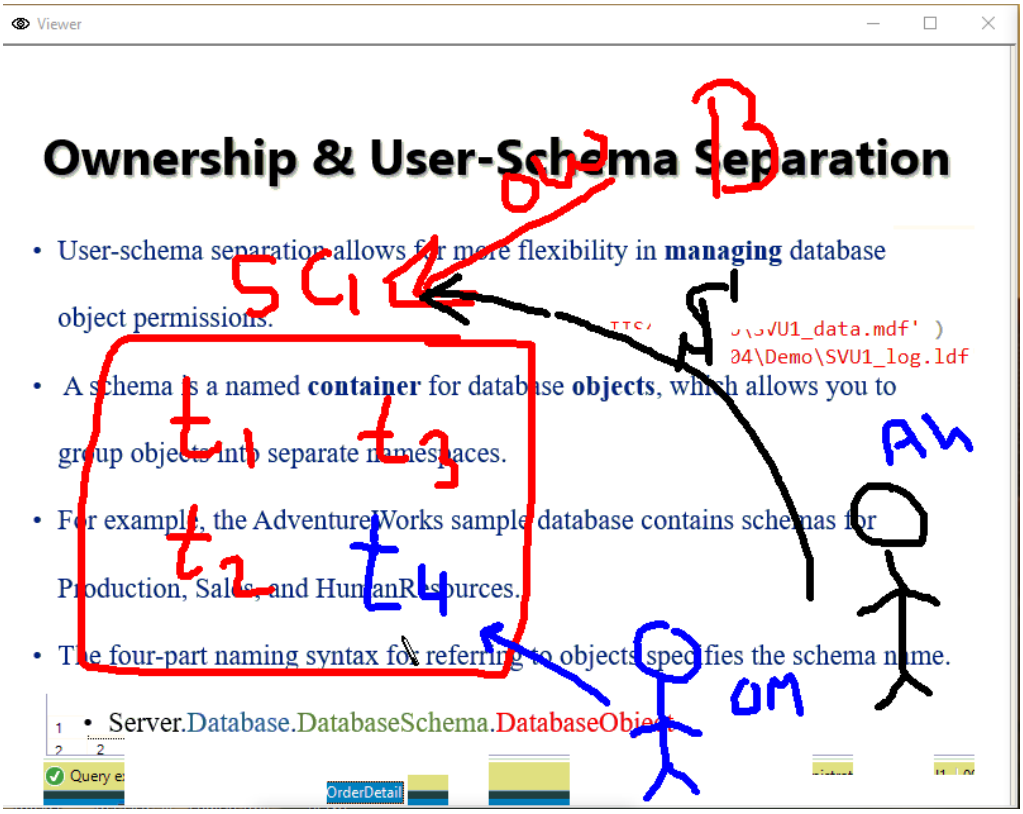
User-schema separation allows for more flexibility in managing database object permissions. A schema is a named container for database objects, which allows you to group objects into separate namespaces. For example, the AdventureWorks sample database contains schemas for Production, Sales, and HumanResources. The four-part naming syntax for referring to objects specifies the schema name.

هرمية التسمية لأي Object هي كالتالي:

*Server.Database.DatabaseSchema.DatabaseObject*

## 5.2. Schema Owners and Permissions

***مثال عن صلاحيات الUsers على ال Schema و الجداول التي بداخلها:***

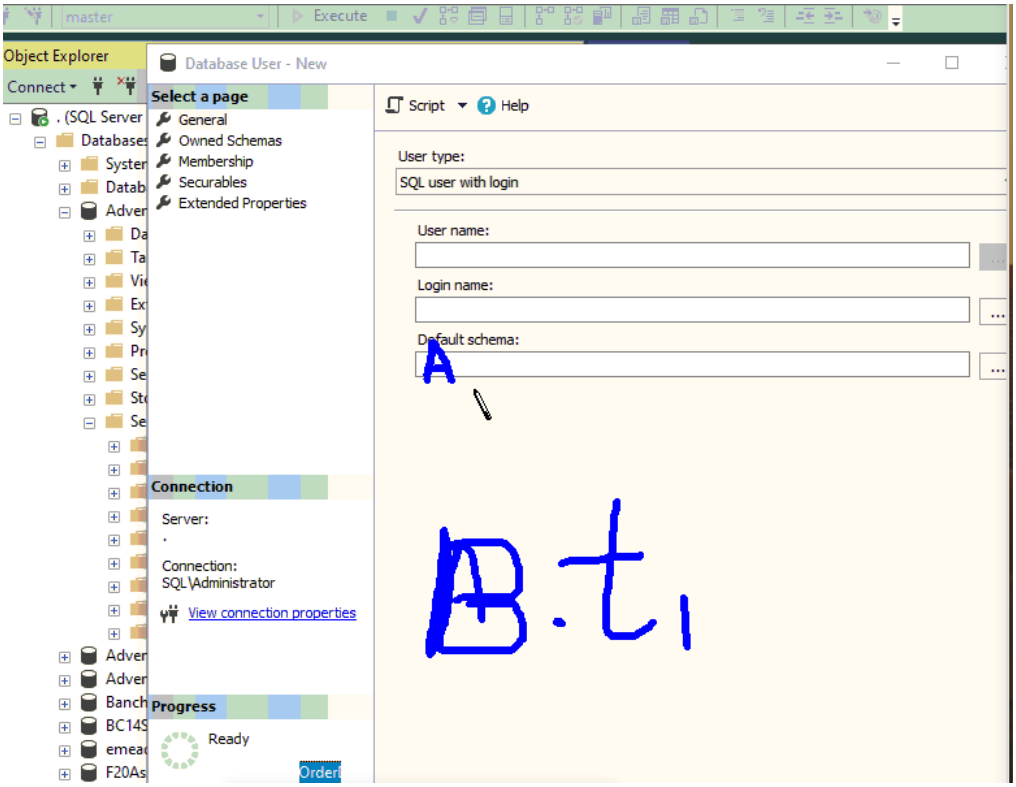



نستطيع بحال وجود ***Schema*** إسمها ***sc1***
تحتوي جداول ***t1,t2,t3,t4***

في حال كان ***Basem*** ال ***Owner*** الخاص بال ***Schema*** ( لديه صلاحيات مطلقة على الجداول الثلاثة )
و كان لدينا ***Ahmad*** نريد إعطائه سماحية قراءة على الجداول, فلدينا طريقتين:
1) نعطيه صلاحية ***Read*** على كل جدول بجدوله
2) نعطيه صلاحية ***Read*** على ال ***Schema***

و كان لدينا ***Omar*** لديه صلاحية كتابة على ال ***Schema*** , فقام بإنشاء جدول ***t4***
سيكون ال ***Owner*** الخاص ب ***t4*** هو ***basem***
(نفس ال ***Owner*** تبع ال ***Schema*** )

و يستطيع ***Basem*** نقل ال ***Ownership*** الخاصة بالجدول ***t4*** إلى ***Omar*** إذا أراد
و سيكون ***Ahmad*** قادر على قراءة جدول ***t4*** في حال تم إعطائه صلاحية قراءة على ال ***Schema*** ( تسمّى هذه االحالة بالتوريث – ***Inheritance*** ) و نستطيع إيقاف خاصيّة الوراثة إذا أردنا بتعليمة ***Deny*** , بينما في حال تم إعطائه سابقا صلاحية قراءة على الثلاث جداول ***t1,t2,t3*** فلن يستطيع قراءة الجدول ***t4***




ال ***Default Schema*** الخاصة بأي ***User*** جديد تسمح بإختصار كتابة ***schema_name.table_name***

و في حال عدم تحديد ال ***Default*** ***Schema*** يدويا, سيتم بعدها إستخدام ***Default*** ***Schema*** عامّة لكل ال ***Users*** و التي يكون إسمها ***dbo schem***
و لا يمكن عمل ***drop*** لها

فنكتبها ***table_name*** وسيتم إستخدام ال ***default schema*** بشكل تلقائي ( ال ***default*** المحددة يدويا, او ال ***dbo*** في حال عدم تحديدها يدويا )

و في حال عدم وجود جدول بذلك الإسم في ال ***Default Schema*** سيتم إعطاء خطأ ***error*** و لن يستمر بالبحث في كل ال ***schemas*** التي ليست ***default***

و في حال وجود صلاحيات لديه على غير ***Schema***, سنحتاج لتحديد إسم ال ***Schema*** قبل ال ***table_name*** مثل: ***B1.t1***

Schemas can be owned by any database principal, and a single principal can own multiple schemas. You can apply security rules to a schema, which are inherited by all objects in the schema. Once you set up access permissions for a schema, those permissions are automatically applied as new objects are added to the schema.

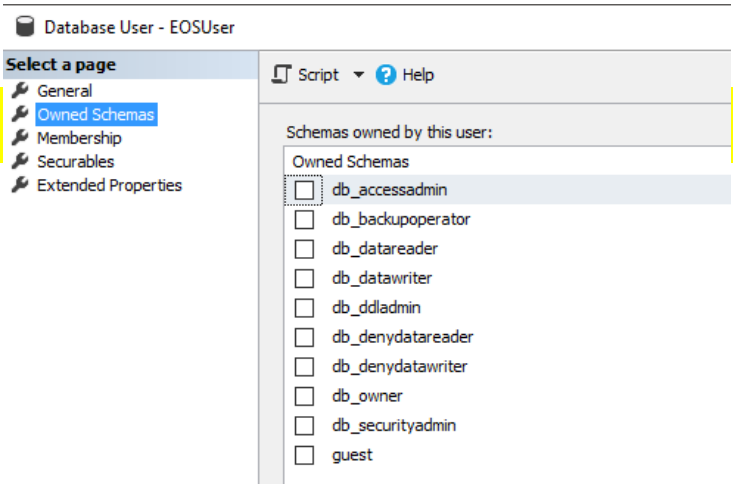
Users can be assigned a default schema, and multiple database users can share the same schema. By default, when developers create objects in a schema, the objects are owned by the security principal that owns the schema, not the developer.

Object ownership can be transferred with ALTER AUTHORIZATION Transact-SQL statement.

A schema can also contain objects that are owned by different users and have more granular permissions than those assigned to the schema.

## 5.3. *Built-In Schemas*

توجد ***schemas*** منشأة بشكل إفتراضي بإسم حسابات مثل:
***db-backupoperator***
و نستطيع القيام ب ***Drop*** لهذه الحسابات ما عدا الحسابات التالية:
***Dbo, guest, sys, information_schema***




SQL Server ships with ten pre-defined schemas that have the same names as the built-in database users and roles. These exist mainly for backward compatibility. You can drop the schemas that have the same names as the fixed database roles if you do not need them. You cannot drop the following schemas:

dbo, guest, sys, INFORMATION_SCHEMA

**The dbo Schema**

**هي ال Default Schema لل user الذي ليس لديه default schema يدويا**

في حال إنشاء **table** داخل **shcema** معيّنة من قبل **user** لم يتم تحديد ال **default schema** ( سيتم إستخدام ال **dbo schema** )
فإن هذا ال **table**

The dbo schema is the default schema for a newly created database. The dbo schema is owned by the dbo user account. By default, users created with the CREATE USER Transact-SQL command have dbo as their default schema.

Users who are assigned the dbo schema do not inherit the permissions of the dbo user account. No permissions are inherited from a schema by users; schema permissions are inherited by the database objects contained in the schema.

When database objects are referenced by using a one-part name, SQL Server first looks in the user's default schema. If the object is not found there, SQL Server looks next in the dbo schema. If the object

is not in the dbo schema, an error is returned.

## sys & INFORMATION_SCHEMA schemas

- لا يمكن إنشاء أي Object على ال Schemas: sys , Information_Schema بالإضافة لعدم القدرة على حذفهم
- لإنتحال شخصية user آخر ( Impersonate ) نستخدم التعليمة:

```
Execute as user=’user_name’;
Go
Select * from shecma_name.table_name;
Go
Revert;
Go
```

و هذه هي الحالة التي يستخدم بها user بدون login
و نستخدم تعليمة revert للخروج من ال user الذي إنتحلنا شخصيته بعد الإنتهاء من إستخدام التعليمة

لإعطاء صلاحية على نستخدم تعليمة grant, و لسحب صلاحية نستخدم تعليمة revoke

They are reserved for system objects. You cannot create objects in these schemas and you cannot drop them.

### *5.4. The Principle of Least Privilege*

إن مصطلح (LUA) least-privileged user account يعني إعطاء كل User أقل ما يمكن من الصلاحيات لملائمة طبيعة عمله, كأسلوب من أساليب الحماية

Developing an application using a least-privileged user account (LUA) approach is an important part of a defensive, in-depth strategy for countering security threats. The LUA approach ensures that users follow the principle of least privilege and always log on with limited user accounts.

Administrative tasks are broken out using fixed server roles, and the use of the sysadmin fixed server role is severely restricted. Always follow the principle of least privilege when granting permissions to database users. Grant the minimum permissions necessary to a user or role to accomplish a given task.

## 6. Permission Statements تعليمات التعامل مع الصلاحيات:

| Permission Statement | Description |
|---|---|
| GRANT | Grants a permission.<br><br>Can assign permissions to a group or role that can be inherited by database users. However, the DENY statement takes precedence over all other permission statements. Therefore, a user who has been denied a permission cannot inherit it from another role. |
| REVOKE | Revokes a permission. This is the default state of a new object. A permission revoked from a user or role can still be inherited from other groups or roles to which the principal is assigned. |

| | |
|---|---|
| DENY | DENY revokes a permission so that it cannot be inherited.<br><br>DENY takes precedence over all permissions, except DENY does not apply to object owners or members of sysadmin.<br><br>If you DENY permissions on an object to the public role it is denied to all users and roles except for object owners and sysadmin members.<br><br>Members of the sysadmin fixed server role and object owners cannot be denied permissions. |

بالإضافة لوجود صلاحية إسمها **With Grant**, تعني صلاحية إعطاء صلاحيات للآخرين.

- في حال الحصول على سماحيات من **user** له صلاحية **With Grant**, فإن هذه السماحيات تكون مؤقتة. حيث تلغى في حال سحب صلاحية ال **With Grant** من ال **User**المعطي.
- في حال الحصول على سماحية **Select** عن طريق **User** له صلاحية **With Grant**, ففي حال سحب صلاحية ال **Select** من ال **User** المعطي ( الذي لديه صلاحية **With Grant** ) ستذهب من ال **User** الثاني
- في حال سحب صلاحية **With Grant** ثم إعادتها لل **User** المعطي, فلن تعود معها كل السماحيات التي تم منحها سابقا عن طريقه لل **Users** الآخرين
- في حال حذف ال **Login** المرتبط بال **User** بصلاحية **With grant** لن يتغير شيئ من الصلاحيات التي قام بإعطائها مسبقا
- لا نستطيع حذف **User** بصلاحية **With Grant** قبل أن نلغي جميع الصلاحيات التي قام بإعطائها
- و لا نستطيع حذف **User** في حال كان **Owner** ل **Schema** إلى أن نسحب منه هذه الصلاحية, و لكن نستطيع حذف ال **Login** الخاص بهذا ال **User** حتى لو لم نسحب منه صلاحية ال **Owner** على ال **Schema** و هذه لن يؤثر على أي صلاحية قام بمنحها

- نستطيع إعطاء صلاحية **Select** ل **User** على جدول معيّن مع منعه من قراءة عامود معيّن من هذا الجدول حيث أن ال **Deny** هو أقوى أوامر الصلاحيات و سيتم تطبيقه

## 7. IMPERSONATE

```
GRANT IMPERSONATE ON USER::[database_principal] TO [database_principal]

GRANT IMPERSONATE ON LOGIN::[server_principal] TO [server_principal]
```

عند منحها ل user, سيستطيع إستخدام التعليمة: Execute as user لتنفيذ تعليمات معيّنة بإستخدام صلاحية User آخر ( إنتحال شخصيته )

و نقوم بالخروج من صلاحيات ال User المنخلة شخصيته, بإستخدام التعليمة: Revert

The IMPERSONATE permission allows the user of the EXECUTE AS statement and also the EXECUTE AS clause to run a stored procedure. This permission can create a complicated administrative environment and should be granted only after you have an understanding of the

implications and potential inappropriate or malicious use. With this permission, it is possible to impersonate a member of the sysadmin role and assume those permissions, so this permission should be granted in controlled scenarios, and perhaps only temporarily.

This permission is most commonly granted for applications that use EXECUTE AS to change their connection security context. You can grant the IMPERSONATE permission on logins or users.

Logins with the CONTROL SERVER permission already have IMPERSONATE ANY LOGIN permission, which should be limited to administrators only. It is unlikely that any application that uses EXECUTE AS would need its service account to have permission to IMPERSONATE any login that currently or ever will exist. Instead, service accounts should be granted IMPERSONATE permissions only for known, appropriate, and approved principals that have been created for the explicit purpose of being impersonated temporarily.

An EXECUTE AS statement should eventually be followed by a REVERT. We use EXECUTE AS and discuss more about this statement later in this chapter.

## 8. CONTROL SERVER|DATABASE

```
GRANT CONTROL SERVER TO [server_principal]

GRANT CONTROL ON DATABASE::[Database_Name] TO [database_principal]
```

هي سماحية على ال Server أو ال DB تعطى من قبل sysadmin, بدون سماحية التعديل على صلاحيات ال sysasdmin

( تستخدم في حال أراد ال sysadmin أخذ إجازة و تسليم المسؤوليات لمساعده, مع حماية نفسه من أن يتم حذف او تعديل حسابه – لن يكون للمستخدم الممنوحة له هذه الصلاحية القدرة على استعمال تعليمة Deny على حساب ال sysadmin)

This effectively grants all permissions on a server or database and is not appropriate for developers or non-administrators.

Granting the CONTROL permission is not exactly the same as granting membership to the sysadmin server role or a db_owner database role, but it has the same effect. Members of the sysadmin role are not affected by DENY permissions, but owners of the CONTROL permission might be.

## 9. Permissions through Procedural Code

- لضمان سماحية تنفيذ أمر ما ( مثل Procedure أو Function ) من قبل مستخدم معيّن بدون أن يكون له أي سماحية قراءة أو كتابة على ال DB أو على ال Object .....

نقوم بإنشاء قناة خاصة لكي يقوم ضمنها بتنفيذ ال Procedure و لكن بشروط.

و لإستعمالها شروط أهمها:

1) عدم التراجع بعد التنفيذ, حيث نستطيع فقط القيام بعملية عكسية لإلغاء بعضهم

2) أن تكون هذه ال Procedure و هذه الجداول لها نفس ال Owner, و في حال ليست لنفس ال Owner سيتم التحقق من ال Underline chains ( من ال Owner الخاص بال Objects )

- Chain: يعني object ضمن Object ضمن .....

مثال عملي عن ال Ownership Chaining:

```
Use master
Drop database if exists svu1
```

```
Create database svu1 on
(name='SVU_Data1', Filename='E\SVU\BAIT\IIS404……\SVU1_Data.mdf')
Log on (Name='SVU1_Log', Filename='E\SVU\BAIT\IIS404….\SVU1_Log')

Use SVU1
```

نقوم بإنشاء Role جديدة و User جديد:

```
Create role sprole;
Go
Create user spuser without login;
Go
```

نقوم بربط ال Role الجديدة مع ال User الجديد:

```
Exec    sp_addrolemember @rolename= 'sprole', @membername='spuser';
Go
```

ننشأ Schema جديدة و ننشأ داخلها Table جديد:

```
Create schema spchema
Go
Create table spschema.sptable (TableID int );
Go
```

ننشأ Procedure داخل ال Schema الجديدة:

```
Create proc spschema.uspselect as begin
Select * from  spschema.sptable
Return
End
```

نقوم بمنح صلاحية للـ Role المنشأة ( بما تتضمّن من Users ) على تنفيذ ال Procedure المنشأة:

```
Grant execute on object :: spschema.uspselect to sprole;
Go
```

نقوم بتجريب تنفيذ تعليمة قراءة بشكل مباشر من الجدول بإستخدام ال User المطبّقة عليه ال Role :

```
Execute as user='spuser';
Go
Select * from spschema.sptable
```

```
Revert
Go
```

النتيجة: لن يتم تنفيذ التعليمة , بسبب عدم إعطائنا صلاحية للـ User بالقراءة بشمل مباشر, بل صلاحية بإستخدام الـ Procedure للقراءة فقط

نقوم الآن بتجربة تنفيذ الـ Procedure بإستخدام الـ User المطبّقة عليه الـ Role:

```
Execute as user=’spuser’;
Go
Exec spschema.uspselect
Go
Revert;
Go
```

النتيجة: سيتم تنفيذ التعليمة بسبب تحقق الشرط ( الـ Owner الخاص بالـ Procedure هو نفسه الـ Owner الخاص بالـ Object ( الـ Table ) حيث أننا عند إنشاء الـ Table و الـ Procedure لم نحدد الـ Owner, و لكن وضعناهم في نفس الـ Schema مما يعني أن لهم نفس الـ Owner )

نقوم بمنع القراءة من الجدول من قبل الـ Role المنشأة, ثم نعيد تنفيذ التعليمة السابقة ( تنفيذ الـ Procedure ) لنرى ما ستكون النتيجة:

```
Deny select on object :: spschema.sptable to sprole

Execute as user=’spuser’;
Go
Exec spschema.uspselect
Go
Revert;
Go
```

النتيجة: سيتم تنفيذ الـ Procedure بسبب أن منع الـ Role من القراءة بشكل مباشر لن يؤثر على وظيفة الـ procedure لأنها لم يكن لها سماحية القراءة بشكل مباشر بالأصل .....

Encapsulating data access through modules such as **stored procedures** and user-defined functions provides an additional layer of protection around your application. You can prevent users from directly interacting with database objects by granting permissions only to stored procedures or functions while denying permissions to underlying objects such as tables. SQL Server achieves this by ownership chaining.

## *9.1. Ownership Chains*

SQL Server ensures that only principals that have been granted permission can access objects. When multiple database objects access each other, the sequence is known as a chain. When SQL Server is traversing the links in the chain, it evaluates permissions differently than it would if it were accessing each item separately.

When an object is accessed through a chain, SQL Server first compares the object's owner to the owner of the calling object (the previous link in the chain). If both objects have the same owner, permissions on the referenced object are not checked.

Whenever an object accesses another object that has a different owner, the ownership chain isbroken and SQL Server must check the caller's security context.

## *9.2. Procedural Code & Ownership Chaining*

Suppose that a user is granted execute permissions on a stored procedure that selects data from a table. If the stored procedure and the table have the same owner, the user doesn't need to be granted

any permissions on the table and can even be denied permissions. However, if the stored procedure and the table have different owners, SQL Server must check the user's permissions on the table before allowing access to the data.

Ownership chaining does not apply in the case of dynamic SQL statements. To call a procedure that executes an SQL statement, the caller must be granted permissions on the underlying tables, leaving your application vulnerable to SQL Injection attack. SQL Server provides new mechanisms, such as impersonation and signing modules with certificates that do not require granting permissions on the underlying tables. These can also be used with CLR stored procedures.

- مجموعة أسئلة إستنتاجية مهمّة للفحص:

## 10. Row-Level Security

Row-Level Security enables customers to control access to rows in a database table based on the characteristics of the user executing a query (for example, group membership or execution context).

ال RLS هي تقنيّة تسمح للمستخدمين بالتحكّم بالوصول إلى أسطر معيّنة من كل جدول بناء على المستخدم الطالب للإستعلام

# Question

- **Can we backup a database in the following scenarios:**
- User has impersonate permissions on db_owner user without having read permissions?
- login has ownership over the db without a db user in it?
- Login has a user that can not read/write but has a db user in backupoperator db role?
- Login with user has execute permission on a stored procedure in db1 which takes backup of db2, discuss this case.

- مثال عملي عن ال RLS:

في حال لدينا Basem في فرع ال HQ, و لدينا أفرع في غير محافظة:

Rola في فرع حمص HM لديها صلاحية على بيانات مبيعات حمص فقط من الTable الموجود في ال HQ

, و Ibrahim في فرع حماة HA لديه صلاحيات على بيانات مبيعات حماة فقط من ال Table الموجود في ال HQ

حيث أننا نريد تطبيق هذا المستوى من الصلاحيات على مستوى قاعدة البيانات و ليس على مستوى ال Application, هذا يسمّى بعملية ال RLS

Row-Level Security (RLS) simplifies the design and coding of security in your application. RLS helps you implement restrictions on data row access. For example, you can ensure that workers access only those data rows that are pertinent to their department, or restrict customers' data accessto only the data relevant to their company.

The access restriction logic is located in the database tier rather than away from the data in another application tier. The database system applies the access restrictions every time that data access is attempted from any tier. This makes your security system more reliable and robust by reducing the surface area of your security system.

Implement RLS by using the CREATE SECURITY POLICY Transact-SQL statement, and predicates created as inline table-valued functions.

RLS supports two types of security predicates.

- ال RLS له نوعان:

1) Filter Predicates: يقوم بفلترة لكل User, و منحه صلاحيّة معيّنة على معلومات بيانات معيّنة من الجدول (يستخدم لمنع قراءة من جدول)

2) Block Predicates: ( يستخدم لمنع الكتابة ), و يقوم بمنع الكتابة حتى لو يوجد قراءة. و يتم منع الكتابة في عدة حالات: ( After Insert, After Update, Before Update, Before Delete )

- Filter predicates silently filter the rows available to read operations (SELECT, UPDATE, and DELETE).
- Block predicates explicitly block write operations (AFTER INSERT, AFTER UPDATE, BEFORE UPDATE, BEFORE DELETE) that violate the predicate.

**مثال:**

**- إذا عندي Douaa هي Sysadmin في ال SQL Server**

**و لدينا مستخدم آخر Rola لديها صلاحية db-owner**

**و لدينا مستخدم آخر Basem لديه صلاحية Sa**

**و لدينا مستخدم آخر Omar لديه صلاحية Owner**

**و لدينا مستخدم آخر Amer مستخدم Filleter Procedure لكي يمنع أي أحد من الإستعلام عن مبيعات حمص مثلا دون أن يعطي أي إستثناءات ....**

**السؤال: أي من السمتخدمين المذكورين سابقا سيستطيع الإستعلام عن مبيعات حمص ؟**

**الجواب: لن يستطيع أي من السمتخدمين المذكروين سابقا الإستعلام عن مبيعات حمص بسبب تطبيق ال Filter Procedure, و لكن جميعهم يستطيعون حذف أو تعديل ال Filter Procedure**

**و لتفعيل إستثناءات على ال Filter Procedure المنشأة نستخدم المثال التالي:**

**Create user Manager without login;**

**Create user sales1 without login;**

**Create user sales2 without login;**

**بعد إنشاء 3 users , نقوم بإنشاء جدول المبيعات: ( المطلوب أن كل User يرى فقط مبيعاته الخاصة, و ال Manager يرى الكل )**

**Create table sales**

**( orderID int,**

**salesRep sysname,**

**Product varchar(10),**

**QTY int**

**);**

**نقوم بإضافة بعض المعلومات على جدول المبيعات:**

**Insert sales values**

**(1, 'sales1' , 'Valve', 5),**

**(2, 'sales1' , 'Wheel', 2),**

**(3, 'sales1' , 'Valve', 4),**

**(4, 'sales2' , 'Bracket', 2),**

**(5, 'sales2' , 'Wheel', 5),**

**(6, 'sales2' , 'Seat', 5),**

**نقوم بالتأكد من محتويات جدول المبيعات:**

**Select * from sales;**

**نقوم بإعطاء صلاحية قراءة من جدول المبيعات للجميع:**

**Grant select on sales to Manager;**

**Grant select on sales to Sales1;**

**Grant select on sales to Sales2;**

**نقوم بإنشاء Schema جديدة**

```
Create schema SecuritySchema;
Go
```

**نقوم بإنشاء Function بحيث تستخدم إسم ال User للفلترة على أساسه مع إعطاء إستثناء لل Manager :**

```
Create function securityschema.fn_SecurityPredicate
(@salesRep as sysname)
Return table
With SchemaBinding
As
    Return select 1 as fn_SecurityPredicate_result
Where @salesRep = User_name() or User_name() = 'Manager';
```

**نقوم بإنشاء Filter Predicate و ربطه مع ال Function المنشأة سابقا و تطبيقه على جدول معيّن ( هنا إخرتنا جدول ال Sales ) : ( مع وضع ال Filter Predicate بحالON )**

```
Create security policy SalesFilter
Add filter predicate SecurityShcema.fn_SecurityPredicate(SalesRep)
On dbo.sales
With (state = ON );
```

**نقوم بالتحقق من عمل ال Filter Predicate مع كل User:**

```
Execute as user = 'Sales1';
Select * from Sales;
Go
```

**ستكون النتيجة بيانات المبيعات الخاصة ب Sales1 فقط**

**Execute as user='Sales2';**

**Select * from Sales;**

**Go**

**ستكون النتيجة بيانات المبيعات الخاصة ب Sales2 فقط**

**Execute as user='Manager';**

**Select * from Sales;**

**Go**

**ستكون النتيجة بيانات المبيعات كلها**

**نقوم بتعطيل ال Filter Predicate ثم نعيد التجربة: ( سيتم السماح لكل User بالإستعلام عن جميع المبيعات )**

Alter Security policy SalesFilter

With (state = OFF);

**Execute as user = 'Sales1';**

**Select * from Sales;**

**Go**

**Execute as user = 'Sales2';**

**Select * from Sales;**

**Go**

**Execute as user = 'Manager';**

**Select * from Sales;**

**Go**

ستكون نتيجة جميع الإستعلامات , عرض جميع المبيعات

Access to row-level data in a table is restricted by a security predicate defined as an inline table- valued function. The function is then invoked and enforced by a security policy. For filter predicates,the application is unaware of rows that are filtered from the result set; if all rows are filtered, then a null set will be returned. For block predicates, any operations that violate the predicate will fail with an error.

Filter predicates are applied while reading data from the base table, and they affect all get operations: SELECT, DELETE (the user cannot delete rows that are filtered), and UPDATE (the user cannot update rows that are filtered, although it is possible to update rows in such way that they will be filtered afterward). Block predicates affect all write operations.

- AFTER INSERT and AFTER UPDATE predicates can prevent users from updating rows to values that violate the predicate.
- BEFORE UPDATE predicates can prevent users from updating rows that currently violate the predicate.
- BEFORE DELETE predicates can block delete operations.

- سيناريوهات مهمّة للإستيعاب:

Both filter and block predicates and security policies have the following behavior:

- You may issue a query against a table that has a security predicate defined but disabled. Any rows that are filtered or blocked are not affected.
- If the dbo user, a member of the db_owner role, or the table owner queries against a table that has a security policy defined and enabled, the rows are filtered or blocked as defined by the security policy.

- Attempts to alter the schema of a table bound by a schema bound security policy will result in an error. However, columns not referenced by the predicate can be altered.
- Attempts to add a predicate on a table that already has one defined for the specified operation (regardless of whether it is enabled or disabled) results in an error.
- For schema bound security policies, attempts to modify a function used as a predicate on a table within a security policy results in an error.
- Defining multiple active security policies that contain non-overlapping predicates, succeeds.

## 11. Transparent Data Encryption (TDE)

**- لحماية قراءة البيانات من ال DB في حال تم الوصول لنسخة إحتياطية منها من قبل مستخدم غير شرعي, نقوم بإستخدام تشفير TDE**

**حيث نقوم بإنشاء Certificate تستخدم مفتاح PKE**

**و نطلب من ال DB أن تشفّر نسخها الإحتياطية ( لن يزداد حجم ملفات النسخ الإحتياطية, و لكن ستتوقف عملية ضغط ملفات النسخ الإحتياطية )**

**و لفك ال Certificate هذه سنحتاج ل Private Key**

Transparent Data Encryption (TDE) encrypts SQL Server, Azure SQL Database, and Azure SQL Data Warehouse data files, known as encrypting data at rest. You can take several precautions tohelp secure the database such as designing a secure system, encrypting confidential assets, and building a firewall around the database servers. However, in a scenario where the physical media (such as drives or backup tapes) are stolen, a malicious party can just restore or attach the database and browse the data. One solution is to encrypt the sensitive data in the database and protect the keysthat are used to encrypt the data with a certificate. This prevents anyone without the keys from using the data, but this kind of protection must be planned in advance.

TDE performs real-time I/O encryption and decryption of the data and log files. The encryption uses a database encryption key (DEK), which is stored in the database boot record for availability during recovery. The DEK is a symmetric key secured by using a certificate stored in the master database of

the server or an asymmetric key protected by an EKM module. TDE protects data "at rest", meaning the data and log files. It provides the ability to comply with many laws, regulations, and guidelines established in various industries. This enables software developers to encrypt data by using AES and 3DES encryption algorithms without changing existing applications.

Encryption of the database file is performed at the page level. The pages in an encrypted database are encrypted before they are written to disk and decrypted when read into memory. TDE does not increase the size of the encrypted database

## *11.1. Using Transparent Data Encryption*

[1].Create a master key

[2].Create or obtain a certificate protected by the master key

[3].Create a database encryption key and protect it by the certificate

[4].Set the database to use encryption.

The following example illustrates encrypting and decrypting the AdventureWorks2012 database using a certificate installed on the server named MyServerCert

```
USE master;
GO
CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<UseStrongPasswordHere>';
go
CREATE CERTIFICATE MyServerCert WITH SUBJECT = 'My DEK Certificate';
go
USE AdventureWorks2012;
GO
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_128
ENCRYPTION BY SERVER CERTIFICATE MyServerCert;
GO
ALTER DATABASE AdventureWorks2012
SET ENCRYPTION ON;
GO
```

## *References*

LeBlanc. P, Assaf. W, Jackson. B, Curnutt M; "**SQL Server 2017 Administration Inside Out**", Microsoft Press (2018).

https://docs.microsoft.com/en-us/azure/sql-database/sql-database-ssms-mfa-authentication

https://docs.microsoft.com/en-us/azure/active-directory/authentication/concept-mfa-howitworks

https://docs.microsoft.com/en-us/sql/ssms/f1-help/connect-to-server-database-engine?view=sql-server-ver15

https://docs.microsoft.com/en-us/sql/t-sql/statements/alter-authorization-transact-sql?view=sql-server-ver15